

# MODEL PARALLELISM: BUILDING AND DEPLOYING LARGE NEURAL NETWORKS

INFERENCE OF LARGE NEURAL NETWORKS



DEEP  
LEARNING  
INSTITUTE

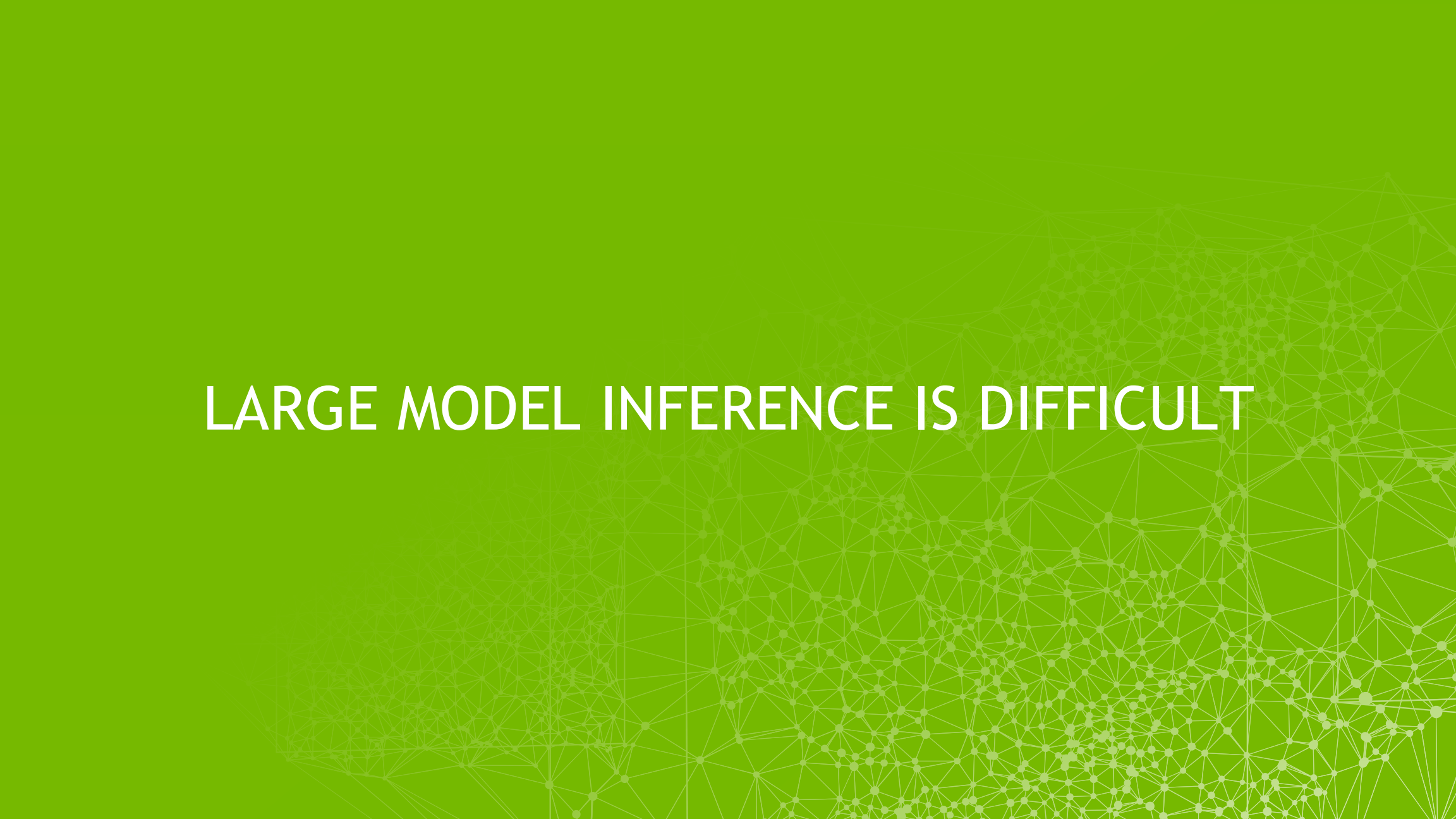
# PART 3

## Inference Of Large Neural Networks Lecture

- Overview of AI Inference Optimization techniques
- Distributed Inference
- TensorRT
- TensorRT-LLM
- Triton Inference Server

### Lab

- Overview of the class environment
- Hugging Face / Pytorch Inference for the LLama2
- Optimize LLama2 with TensorRT-LLM
- Deploy LLama2 with Triton Inference Server



LARGE MODEL INFERENCE IS DIFFICULT

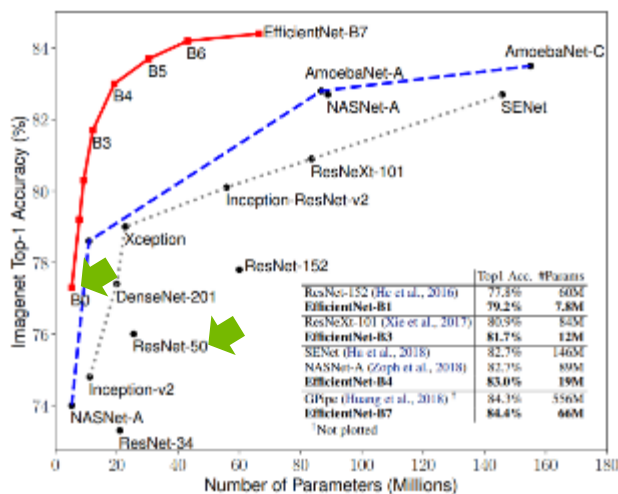


LET'S DIVE INTO THE DETAILS

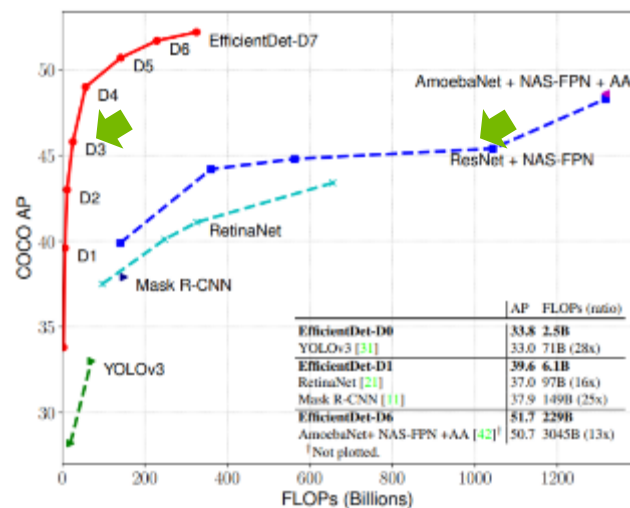
# MODEL SELECTION

Not all models are created equally

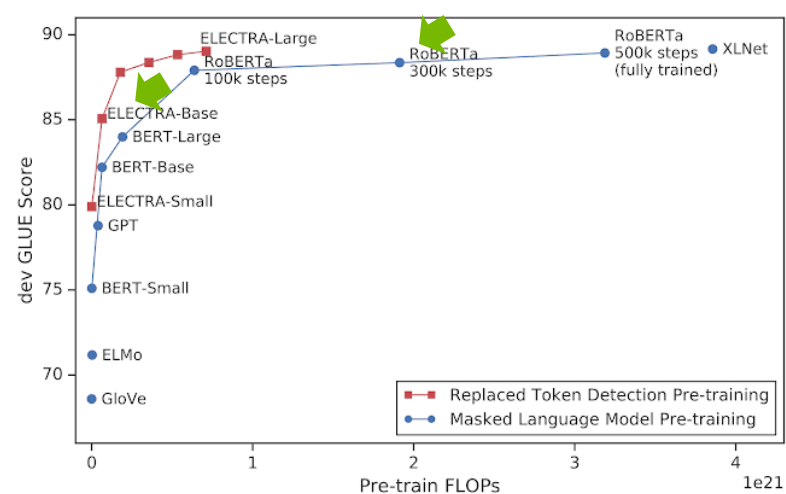
Image Classification



Object detection



NLP



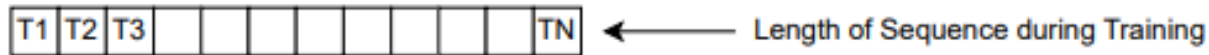
<https://bair.berkeley.edu/blog/2020/03/05/compress/>

Li, Z., Wallace, E., Shen, S., Lin, K., Keutzer, K., Klein, D., & Gonzalez, J. E. (2020). Train large, then compress: Rethinking model size for efficient training and inference of transformers. arXiv preprint arXiv:2002.11794.



# MODEL SELECTION

## Sequence length



Interpolated Sequences can cause errors in accuracy and perplexity

Perplexity (lower is better) - How confused is the model by the input

Larger sequence at training = higher training costs

# MODEL SELECTION

## Sequence length

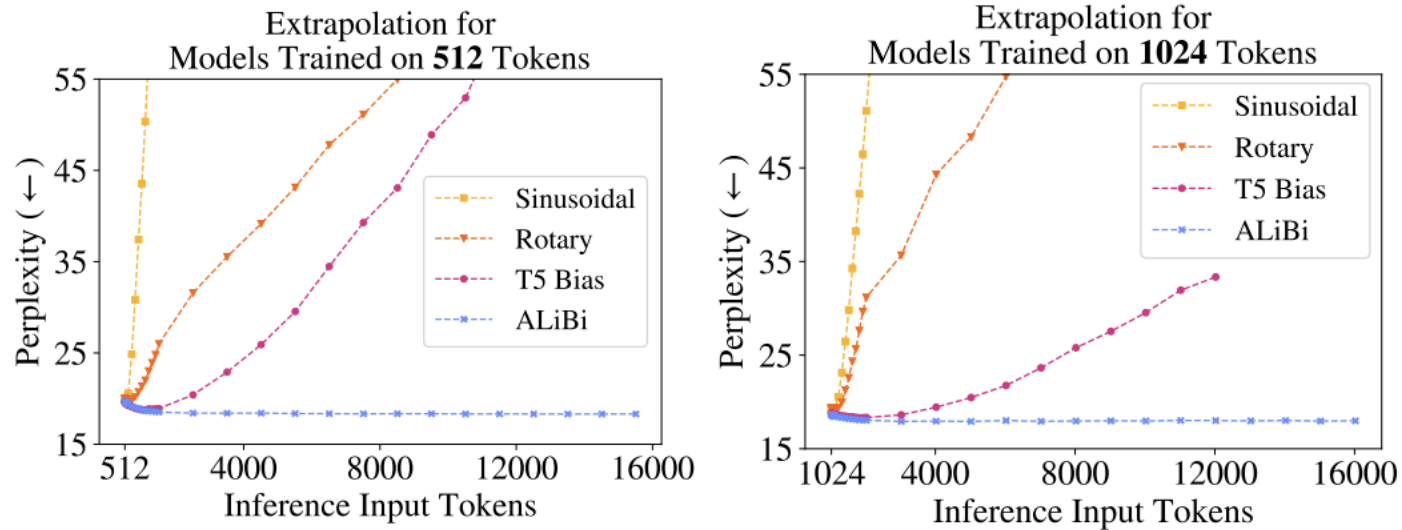
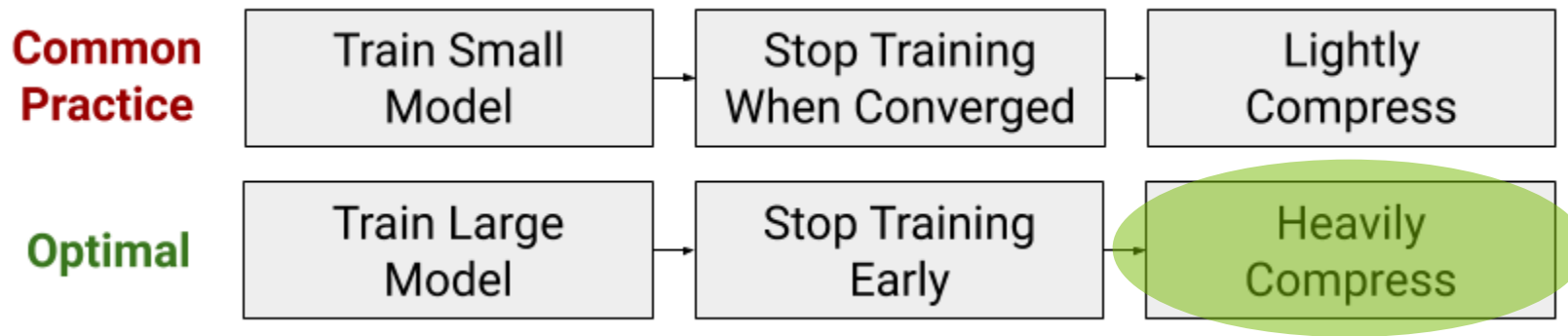


Figure 1: Extrapolation: as the (validation-set's) input sequence gets longer ( $x$ -axis), current position methods (sinusoidal, rotary, and T5) show degraded perplexity ( $y$ -axis, lower is better), but our method (§3) does not. Models were trained on WikiText-103 with sequences of  $L = 512$  (left) or  $L = 1,024$  (right) tokens. T5 ran out of memory on our 32GB GPU. For more detail on exact perplexities and runtimes, see Tables 2 and 3 in the appendix.

# INCREASING IMPORTANCE OF PRUNING AND QUANTIZATION

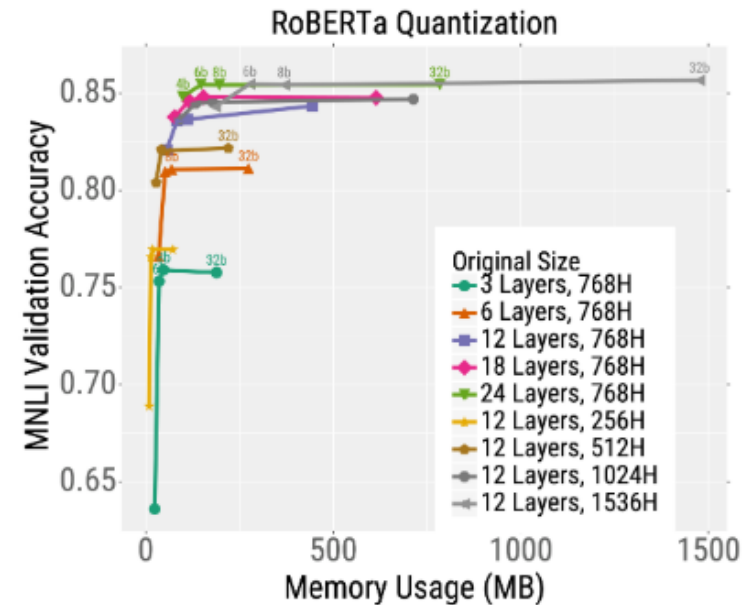
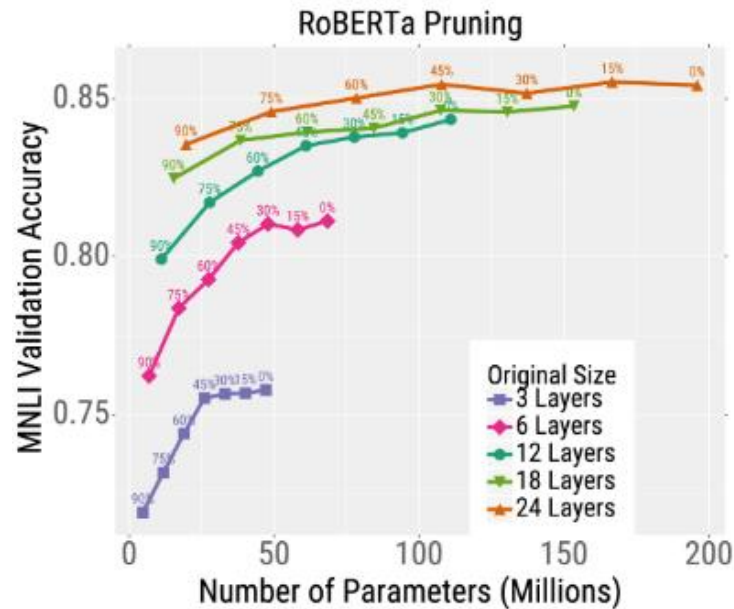
E.g. Train Large then compress





# MODEL SELECTION

Not all models respond in the same way to knowledge distillation, pruning and quantization



<https://bair.berkeley.edu/blog/2020/03/05/compress/>

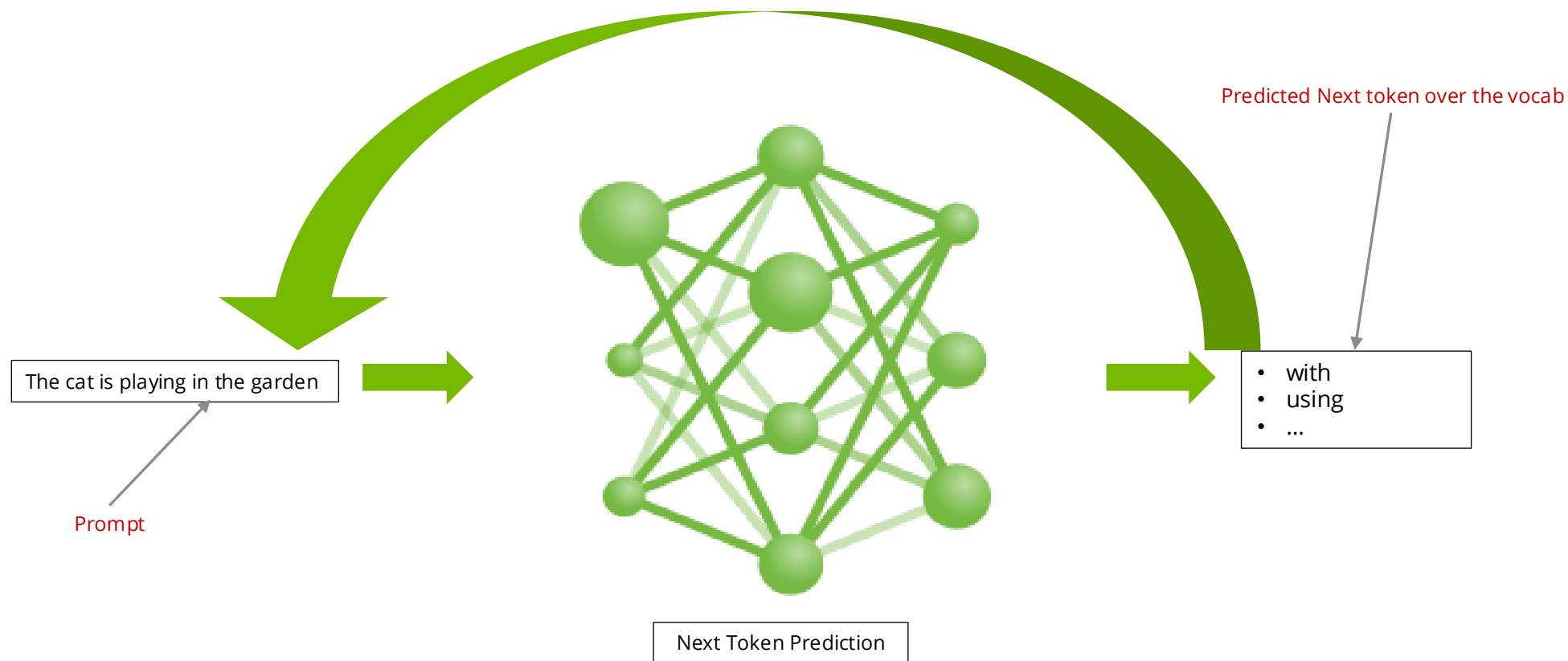
Li, Z., Wallace, E., Shen, S., Lin, K., Keutzer, K., Klein, D., & Gonzalez, J. E. (2020). Train large, then compress: Rethinking model size for efficient training and inference of transformers. arXiv preprint arXiv:2002.11794.

# AUTOREGRESSIVE MODELS

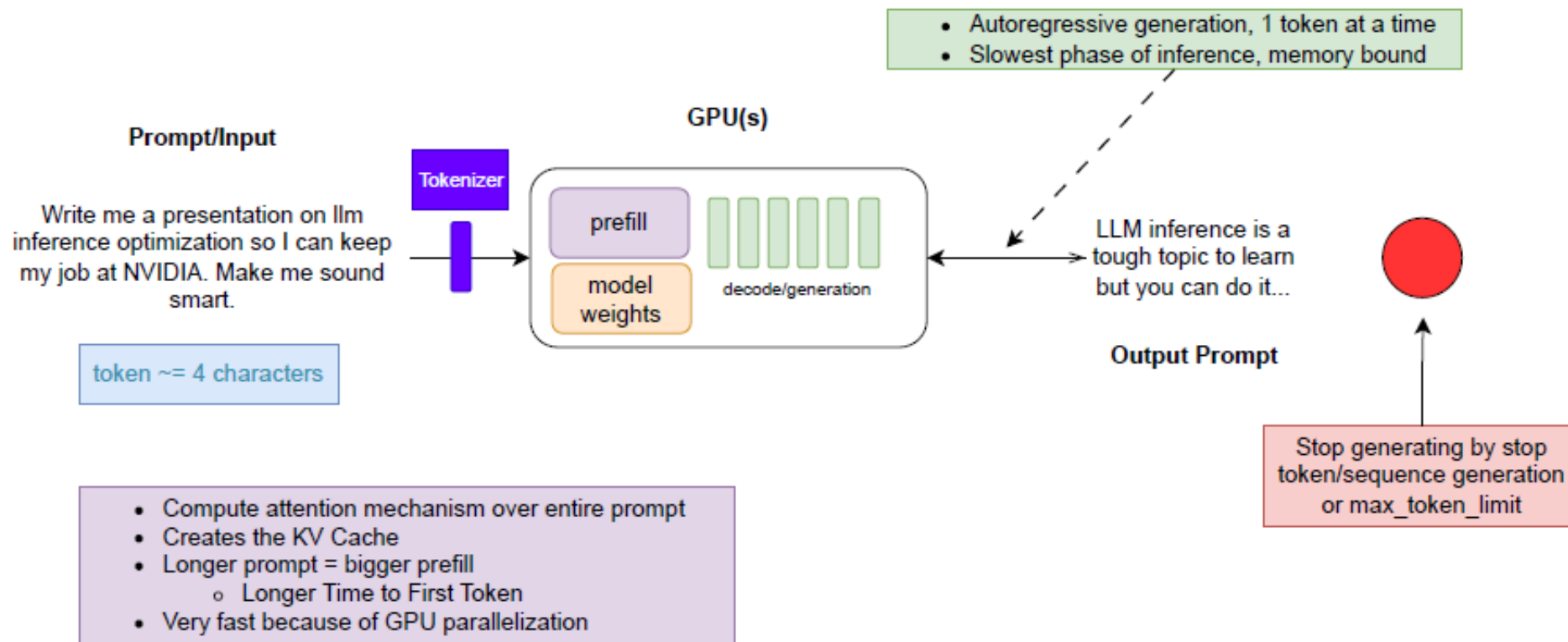


# LARGE LANGUAGE MODEL

Auto-Regressive Loop

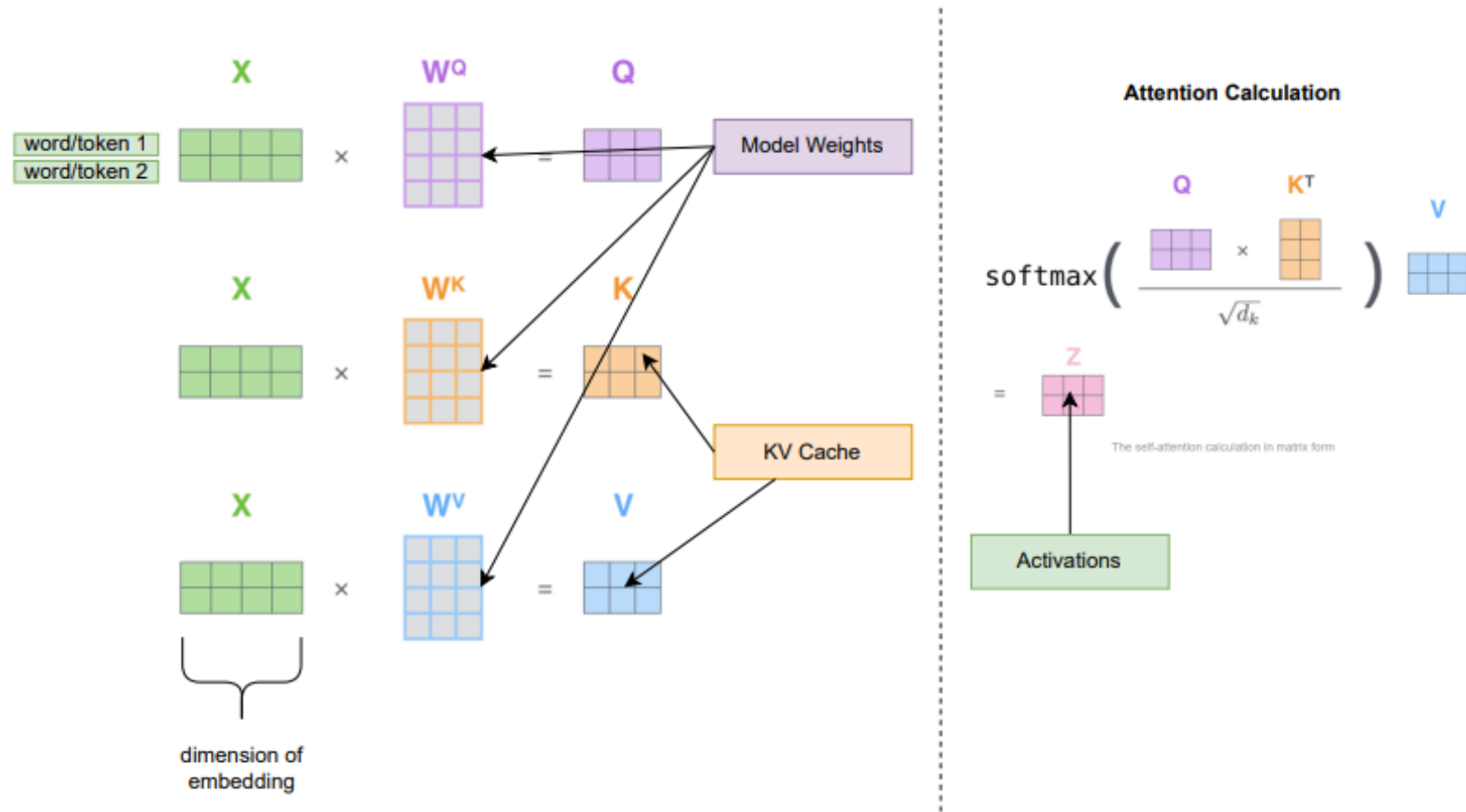


# UNDERSTANDING LLM INFERENCE



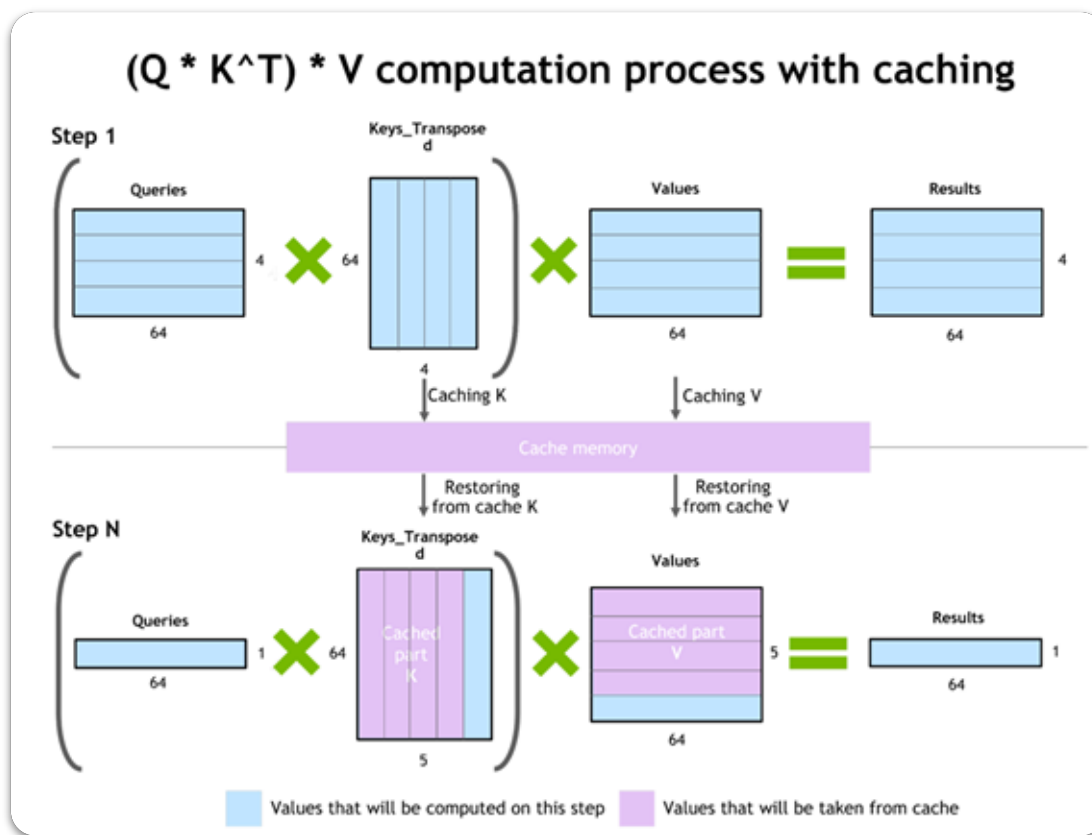
Source: [Mastering LLM Techniques: Inference Optimization](#)

# GPU MEMORY, MODEL WEIGHTS, KV CACHE AND ATTENTION



Source: [Jay Alammar: Illustrated Transformer](#)

# AUTOREGRESSIVE MODELS - KV CACHING



Caching mechanism in NVIDIA TensorRT-LLM (formerly FasterTransformer)





GPU MEMORY IS A PRECIOUS RESOURCE

SAVE AS MUCH AS YOU CAN

# QUANTIZATION

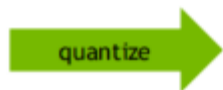
The background of the slide features a smooth gradient transitioning from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of small white dots connected by thin white lines, creating a mesh-like or molecular structure that is more prominent on the right side of the image.

# QUANTIZATION

The idea

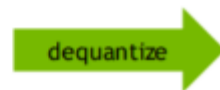
|      |      |      |
|------|------|------|
| 0.34 | 3.75 | 5.64 |
| 1.12 | 2.7  | -0.9 |
| -4.7 | 0.68 | 1.43 |

FP32  
(pre-quantized)



|    |     |     |
|----|-----|-----|
| 64 | 134 | 217 |
| 76 | 119 | 21  |
| 3  | 81  | 99  |

INT8  
(quantized)



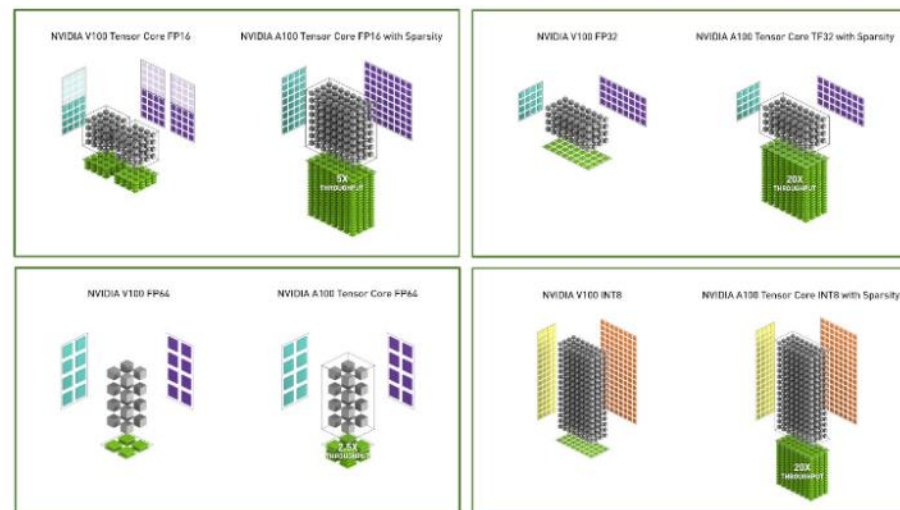
|      |      |       |
|------|------|-------|
| 0.41 | 3.62 | 5.29  |
| 1.3  | 2.8  | -0.92 |
| -4.5 | 0.71 | 1.39  |

FP32  
(dequantized)

# QUANTIZATION

## The rationale

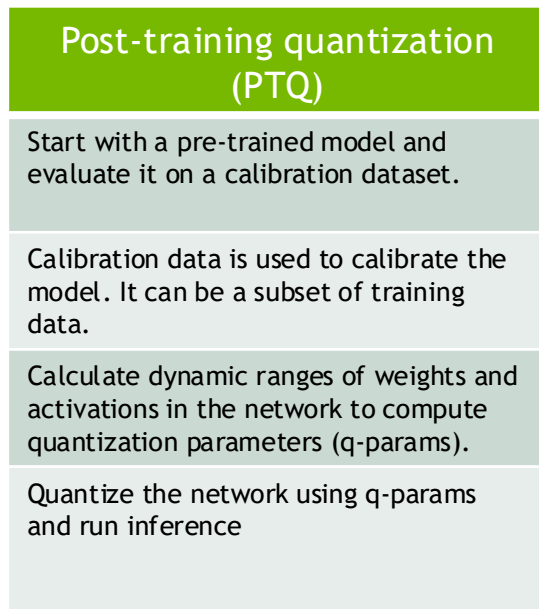
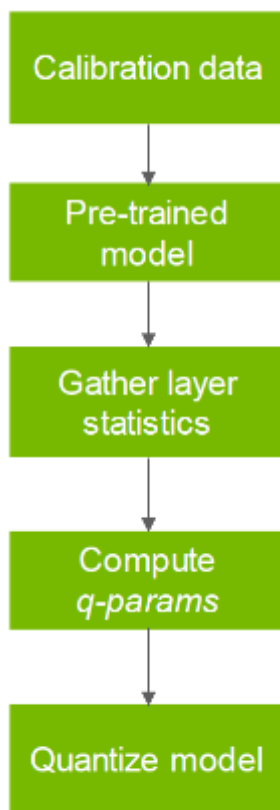
| Input Datatype | Accumulation Datatype | Math Throughput | Bandwidth Reduction |
|----------------|-----------------------|-----------------|---------------------|
| FP32           | FP32                  | 1x              | 1x                  |
| FP16           | FP16                  | 8x              | 2x                  |
| INT8           | INT32                 | 16x             | 4x                  |
| INT4           | INT32                 | 32x             | 8x                  |
| INT1           | INT32                 | 128x            | 32x                 |



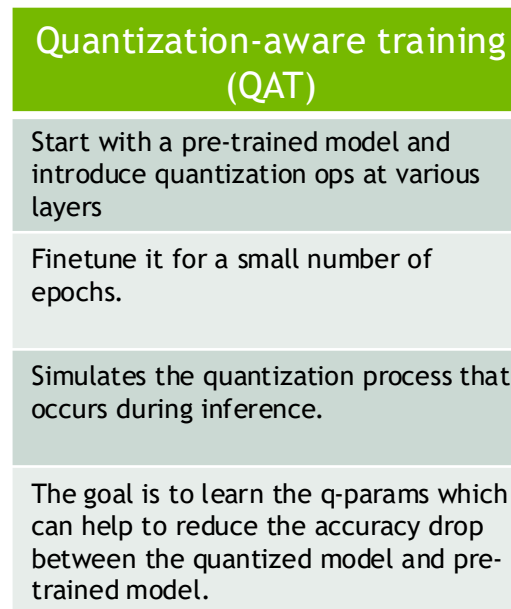
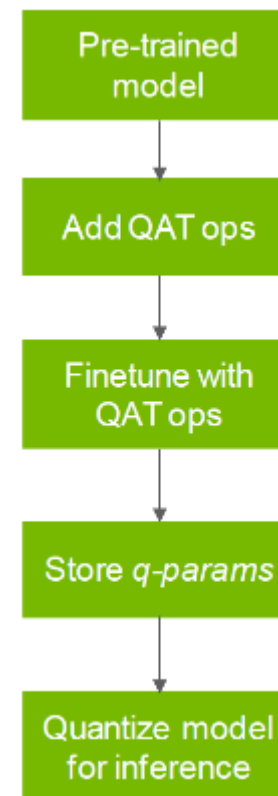
# QUANTIZATION

## Approaches

### Post-training quantization(PTQ)

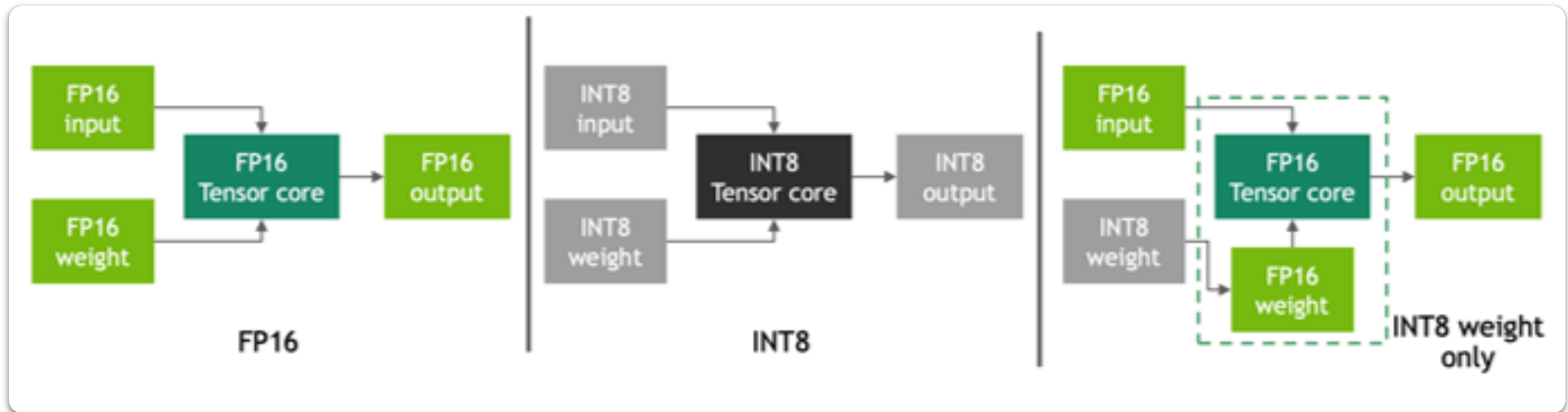


### Quantization-aware training (QAT)



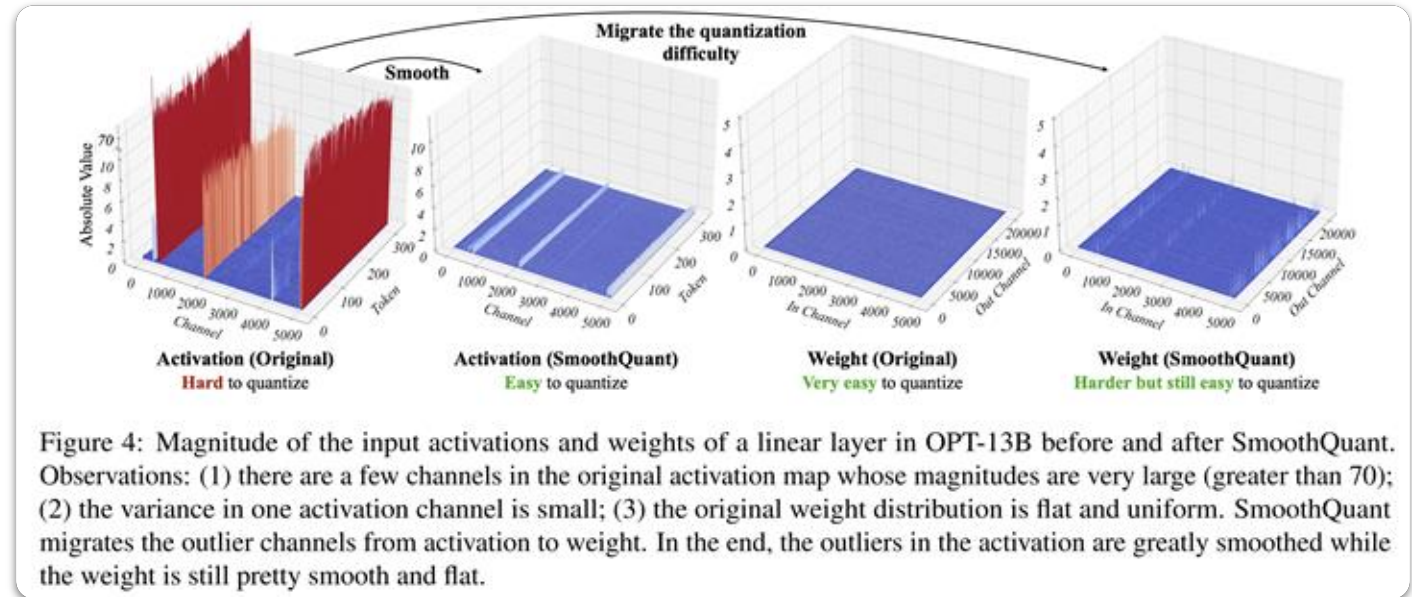
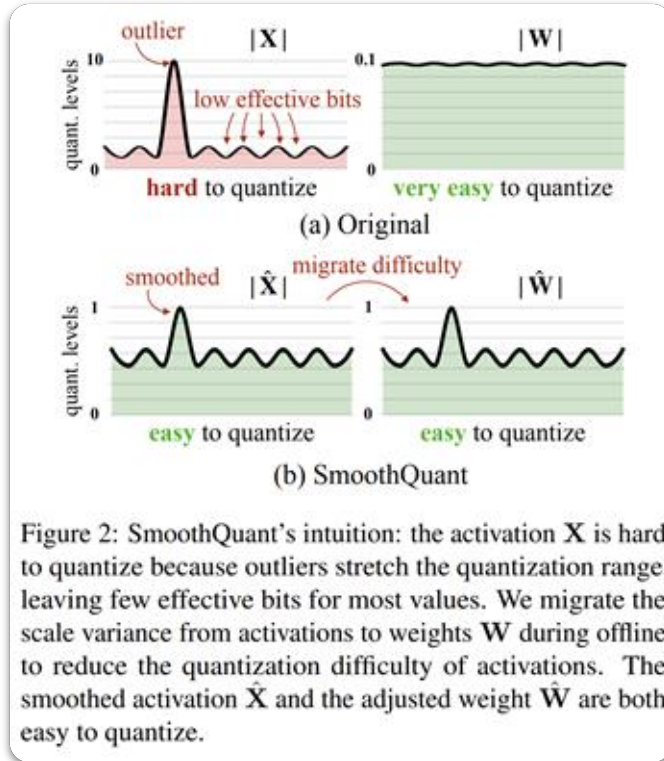
# INT8

Several options



# INT8

## Smooth quantization





# INT8

## Smooth quantization

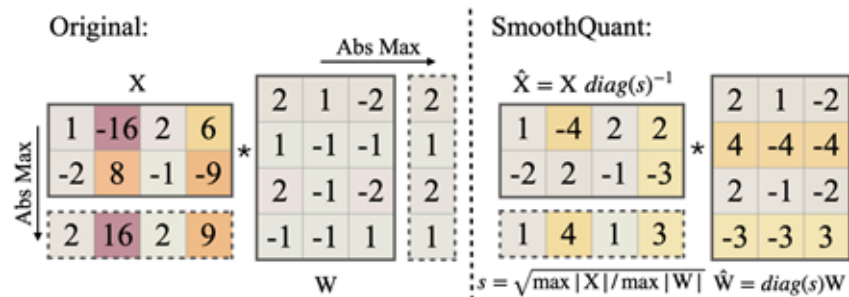


Figure 5: Main idea of SmoothQuant when  $\alpha$  is 0.5. The smoothing factor  $s$  is obtained on calibration samples and the entire transformation is performed offline. At runtime, the activations are smooth without scaling.

Table 4: SmoothQuant works for different LLMs. We can quantize the 3 largest, openly available LLM models into INT8 without degrading the accuracy. For OPT-175B and BLOOM-176B, we show the average accuracy on WinoGrande, HellaSwag, PIQA, and LAMBADA. For GLM-130B we show the average accuracy on LAMBADA, MMLU, MNLI, and QNLI. \*Accuracy is not column-wise comparable due to different datasets.

| Method              | OPT-175B     | BLOOM-176B   | GLM-130B*    |
|---------------------|--------------|--------------|--------------|
| FP16                | 71.6%        | 68.2%        | 73.8%        |
| W8A8                | 32.3%        | 64.2%        | 26.9%        |
| ZeroQuant           | 31.7%        | 67.4%        | 26.7%        |
| LLM.int8()          | 71.4%        | 68.0%        | 73.8%        |
| Outlier Suppression | 31.7%        | 54.1%        | 63.5%        |
| SmoothQuant-O1      | <b>71.2%</b> | 68.3%        | <b>73.7%</b> |
| SmoothQuant-O2      | 71.1%        | <b>68.4%</b> | 72.5%        |
| SmoothQuant-O3      | 71.1%        | 67.4%        | 72.8%        |

# INT8

## Smooth quantization - Implication on inference

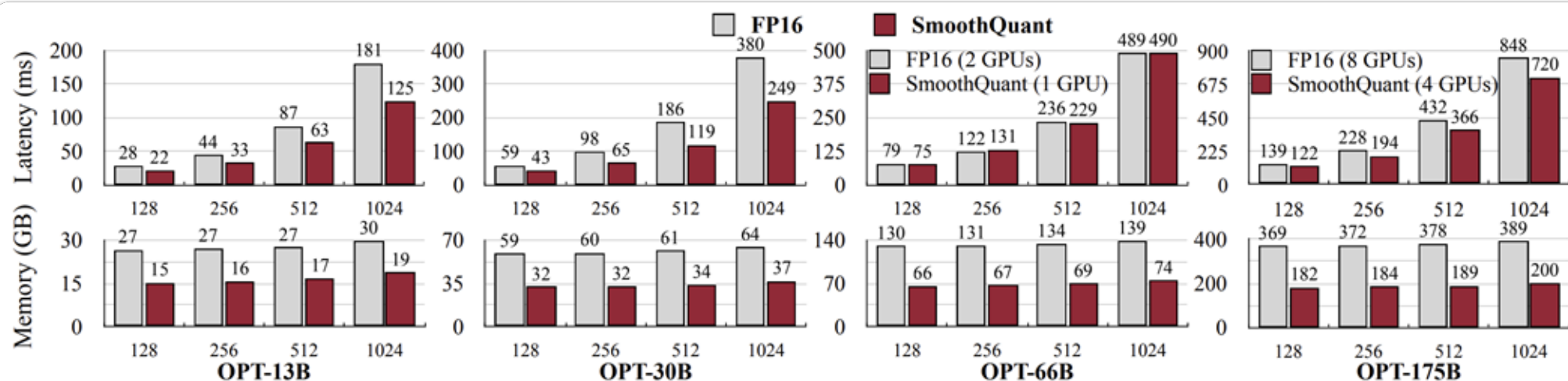


Figure 9: Inference latency (top) and memory usage (bottom) of the FasterTransformer implementation on NVIDIA A100-80GB GPUs. For smaller models, the latency can be significantly reduced with SmoothQuant-O3 by up to 1.56x compared to FP16. For the bigger models (OPT-66B and 175B), we can achieve similar or even faster inference using only **half** number of GPUs. Memory footprint is almost halved compared to FP16.



CAN WE DO BETTER?

# NOT JUST THEORY

Significant success of lower precision inference

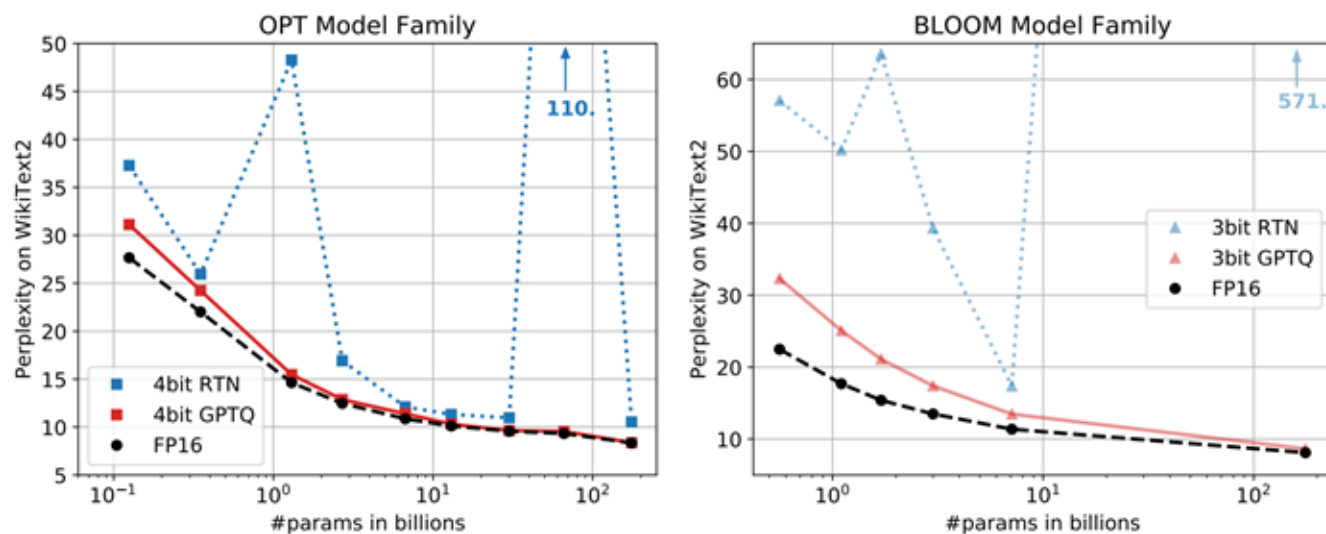
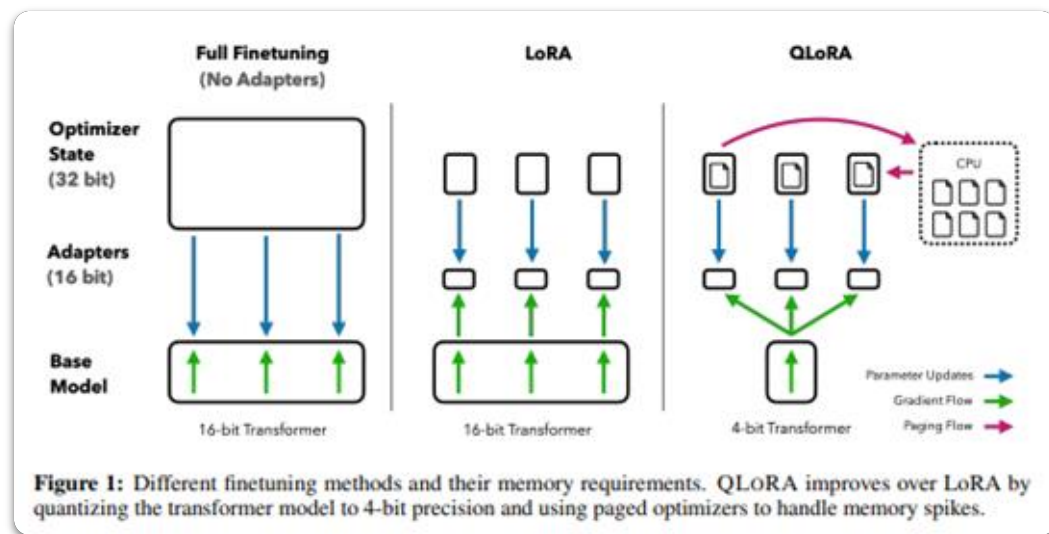


Figure 1: Quantizing OPT models to 4 and BLOOM models to 3 bit precision, comparing GPTQ with the FP16 baseline and round-to-nearest (RTN) (Yao et al., 2022; Dettmers et al., 2022).

# NOT JUST THEORY

Significant success of lower precision inference



**Table 6:** Zero-shot Vicuna benchmark scores as a percentage of the score obtained by ChatGPT evaluated by GPT-4. We see that OASST1 models perform close to ChatGPT despite being trained on a very small dataset and having a fraction of the memory requirement of baseline models.

| Model / Dataset  | Params | Model bits | Memory | ChatGPT vs Sys | Sys vs ChatGPT | Mean          | 95% CI |
|------------------|--------|------------|--------|----------------|----------------|---------------|--------|
| GPT-4            | -      | -          | -      | 119.4%         | 110.1%         | <b>114.5%</b> | 2.6%   |
| Bard             | -      | -          | -      | 93.2%          | 96.4%          | 94.8%         | 4.1%   |
| <b>Guanaco</b>   | 65B    | 4-bit      | 41 GB  | 96.7%          | 101.9%         | <b>99.3%</b>  | 4.4%   |
| Alpaca           | 65B    | 4-bit      | 41 GB  | 63.0%          | 77.9%          | 70.7%         | 4.3%   |
| FLAN v2          | 65B    | 4-bit      | 41 GB  | 37.0%          | 59.6%          | 48.4%         | 4.6%   |
| <b>Guanaco</b>   | 33B    | 4-bit      | 21 GB  | 96.5%          | 99.2%          | <b>97.8%</b>  | 4.4%   |
| Open Assistant   | 33B    | 16-bit     | 66 GB  | 91.2%          | 98.7%          | 94.9%         | 4.5%   |
| Alpaca           | 33B    | 4-bit      | 21 GB  | 67.2%          | 79.7%          | 73.6%         | 4.2%   |
| FLAN v2          | 33B    | 4-bit      | 21 GB  | 26.3%          | 49.7%          | 38.0%         | 3.9%   |
| Vicuna           | 13B    | 16-bit     | 26 GB  | 91.2%          | 98.7%          | <b>94.9%</b>  | 4.5%   |
| <b>Guanaco</b>   | 13B    | 4-bit      | 10 GB  | 87.3%          | 93.4%          | 90.4%         | 5.2%   |
| Alpaca           | 13B    | 4-bit      | 10 GB  | 63.8%          | 76.7%          | 69.4%         | 4.2%   |
| HH-RLHF          | 13B    | 4-bit      | 10 GB  | 55.5%          | 69.1%          | 62.5%         | 4.7%   |
| Unnatural Instr. | 13B    | 4-bit      | 10 GB  | 50.6%          | 69.8%          | 60.5%         | 4.2%   |
| Chip2            | 13B    | 4-bit      | 10 GB  | 49.2%          | 69.3%          | 59.5%         | 4.7%   |
| Longform         | 13B    | 4-bit      | 10 GB  | 44.9%          | 62.0%          | 53.6%         | 5.2%   |
| Self-Instruct    | 13B    | 4-bit      | 10 GB  | 38.0%          | 60.5%          | 49.1%         | 4.6%   |
| FLAN v2          | 13B    | 4-bit      | 10 GB  | 32.4%          | 61.2%          | 47.0%         | 3.6%   |
| <b>Guanaco</b>   | 7B     | 4-bit      | 5 GB   | 84.1%          | 89.8%          | <b>87.0%</b>  | 5.4%   |
| Alpaca           | 7B     | 4-bit      | 5 GB   | 57.3%          | 71.2%          | 64.4%         | 5.0%   |
| FLAN v2          | 7B     | 4-bit      | 5 GB   | 33.3%          | 56.1%          | 44.8%         | 4.0%   |



# PRUNING

The background of the slide features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a mesh-like or molecular structure that spans the entire width of the image.

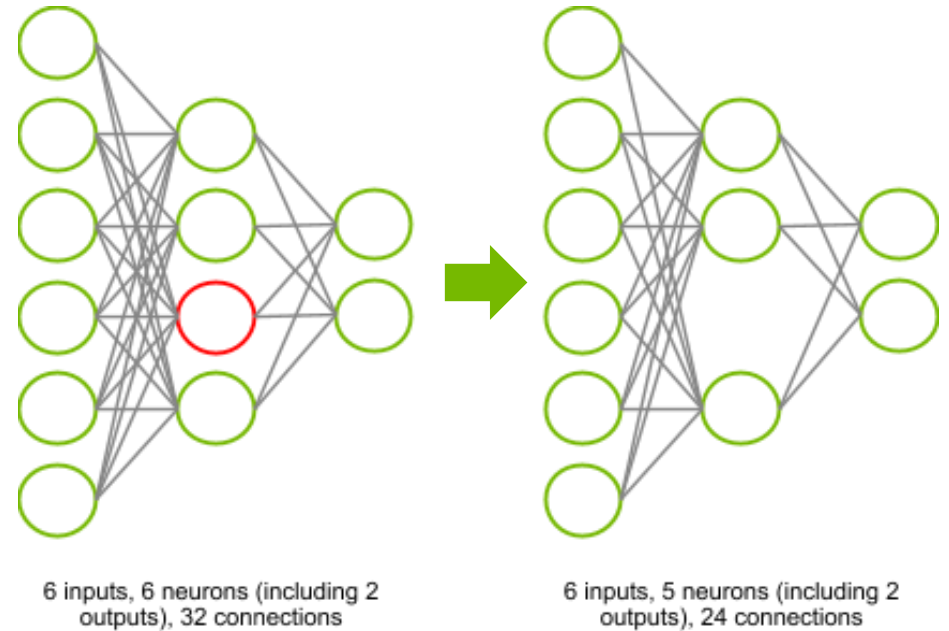
# MODEL OPTIMIZATION

## Pruning

Reduce the complexity of neural networks by removing unnecessary connections

- Reduce memory bandwidth
- Reduce memory footprint
- Accelerate the compute

Challenge: Maintain accuracy of the original unpruned network





# STRUCTURED SPARSITY SUPPORT

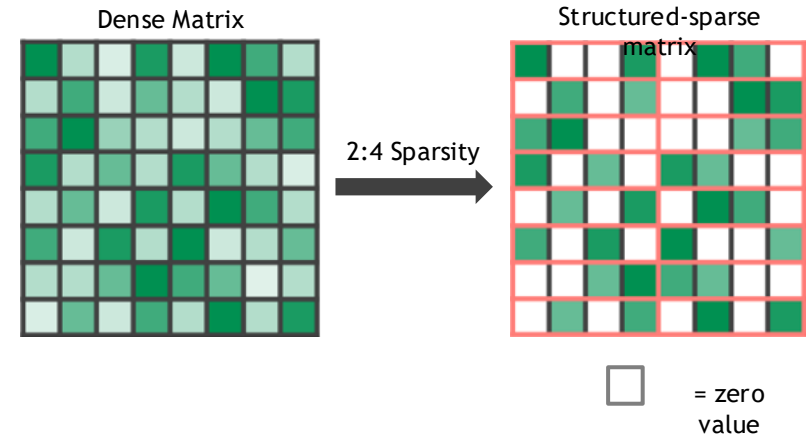
Accelerate Inference with Sparse Tensor Core (starting from Ampere)

Maximize throughput at low latency with sparsity

New optimizations with 2:4 fine-grained structured sparsity result in greater performance reducing the weights in half

ASP (Automatic SParsity) provides easy-to-use workflow to induce the sparsity while maintaining accuracy of original dense network

TensorRT accelerates inference using sparse kernels



```
# Import ASP (Automatic SParsity)
from apex.contrib.sparsity import ASP

# In training phase, to augment the model and the
optimizer #for sparse training/inference
ASP.prune_trained_model(model, optimizer)
```

Training Phase

Enable Sparsity by setting the `kSPARSE_WEIGHTS` flag in `IBuilderConfig`

Inference Phase

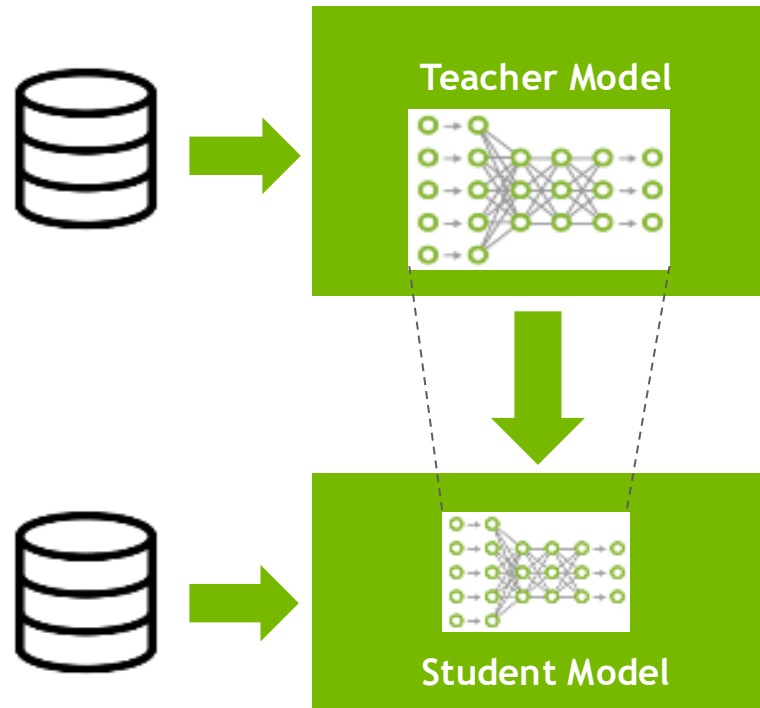
<https://developer.nvidia.com/blog/structured-sparsity-in-the-nvidia-ampere-architecture-and-applications-in-search-engines/>

# DISTILLATION

The background of the slide features a smooth gradient transitioning from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small dots, creating a mesh-like or molecular structure that is denser on the right side.

# KNOWLEDGE DISTILLATION

The idea



# KNOWLEDGE DISTILLATION

## DistillBERT

Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

| Model      | Score | CoLA | MNLI | MRPC | QNLI | QQP  | RTE  | SST-2 | STS-B | WNLI |
|------------|-------|------|------|------|------|------|------|-------|-------|------|
| ELMo       | 68.7  | 44.1 | 68.6 | 76.6 | 71.1 | 86.2 | 53.4 | 91.5  | 70.4  | 56.3 |
| BERT-base  | 79.5  | 56.3 | 86.7 | 88.6 | 91.8 | 89.6 | 69.3 | 92.7  | 89.0  | 53.5 |
| DistilBERT | 77.0  | 51.3 | 82.2 | 87.5 | 89.2 | 88.5 | 59.9 | 91.3  | 86.9  | 56.3 |

Table 2: **DistilBERT yields to comparable performance on downstream tasks.** Comparison on downstream tasks: IMDb (test accuracy) and SQuAD 1.1 (EM/F1 on dev set). D: with a second step of distillation during fine-tuning.

| Model          | IMDb (acc.) | SQuAD (EM/F1) |
|----------------|-------------|---------------|
| BERT-base      | 93.46       | 81.2/88.5     |
| DistilBERT     | 92.82       | 77.7/85.8     |
| DistilBERT (D) | -           | 79.1/86.9     |

Table 3: **DistilBERT is significantly smaller while being constantly faster.** Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

| Model      | # param. (Millions) | Inf. time (seconds) |
|------------|---------------------|---------------------|
| ELMo       | 180                 | 895                 |
| BERT-base  | 110                 | 668                 |
| DistilBERT | 66                  | 410                 |

*DistillBERT retain 97% of BERT performance while is only 66M parameters*

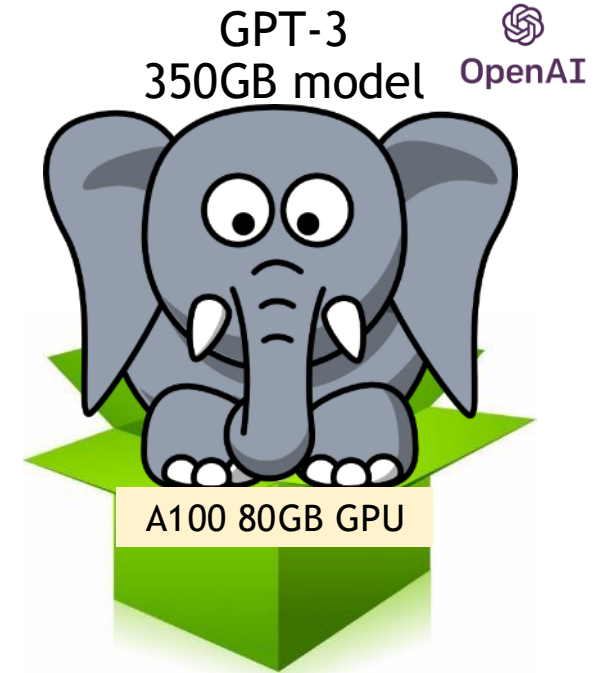
# INFERENCE OF HUGE MODELS

The background of the slide features a smooth gradient transitioning from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a mesh-like or molecular structure that spans the entire width of the image.

# INFERENCE OF HUGE MODELS

## Goals and Challenges

- **Goal:** To infer huge models in an efficient and convenient way, including
  - Maximizing Utilization of GPUs
  - A unified and simple inference solution for many models in production
  - Easier deployments, scaling and support
  - Maximizing Throughput, Minimizing Latency
- **Challenges:**
  - Huge model requires more memory than available on 1 GPU
  - Model needs to be optimized before the inference
  - Frameworks used for training Huge Models are quite complex and inadequate for inference



# PRODUCTION DEPLOYMENT

## Executive Math

175B parameters

2 bytes per weight  
(FP16)



$$\frac{175 * 10^9 * 2}{1024 * 1024 * 1024} = 325.96GB$$

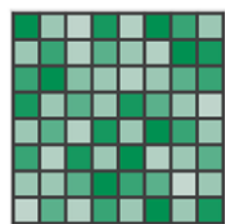


We want GPU memory  
consumption in GB



# PRODUCTION DEPLOYMENT

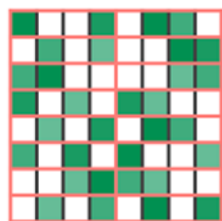
## Pruning - 2:4 Structured Sparsity



Dense Matrix



Fine-grained  
structured  
sparsity



Sparse Matrix

175B parameters



2 bytes per weight  
(FP16)



$$\frac{175 * 10^9 * 2}{1024 * 1024 * 1024 * 2} = 162.98GB$$



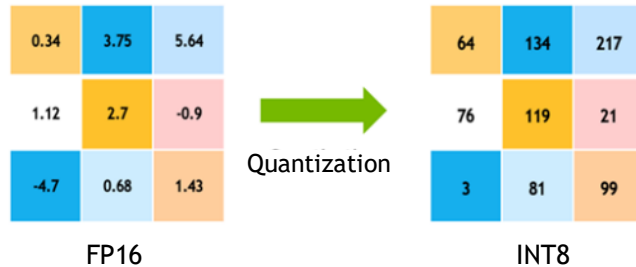
We want GPU memory  
consumption in GB



Remove half of  
the weights

# PRODUCTION DEPLOYMENT

## Quantization



175B parameters

1 byte per weight (INT8)

$$\frac{175 * 10^9 * 1}{1024 * 1024 * 1024 * 2} = 81.49GB$$

We want GPU memory consumption in GB

Remove half of the weights

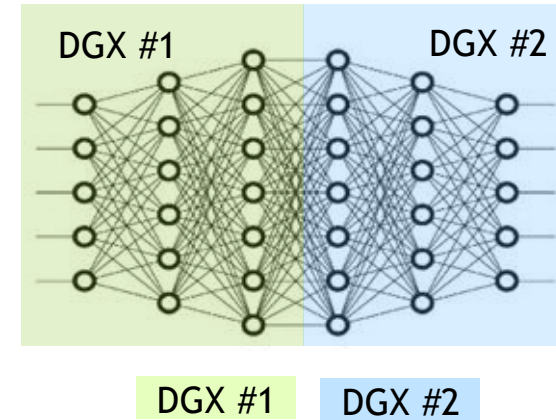
# DISTRIBUTED INFERENCE

The background of the slide is a solid green color. Overlaid on this background is a complex, white network diagram. This diagram consists of numerous small circular nodes connected by thin, white lines, creating a web-like structure that resembles a distributed system or a neural network. The network is more densely packed on the right side of the slide and becomes sparser towards the left.

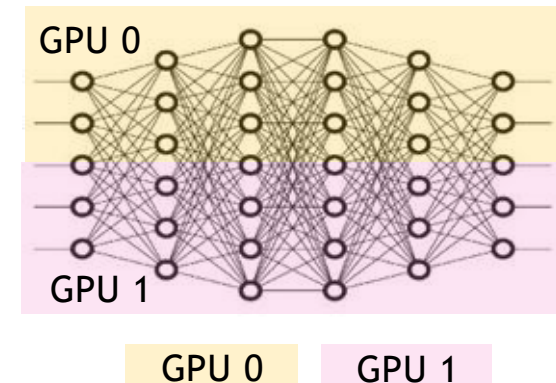
# MODEL PARALLELISM

## Complementary Types of Model Parallelism

- Inter-Layer (Pipeline) Parallelism
  - Split sets of layers across multiple devices
  - **Inference:**
    - *Maximizes GPU utilization and Throughput*
    - *Can be used easily with TRITON*

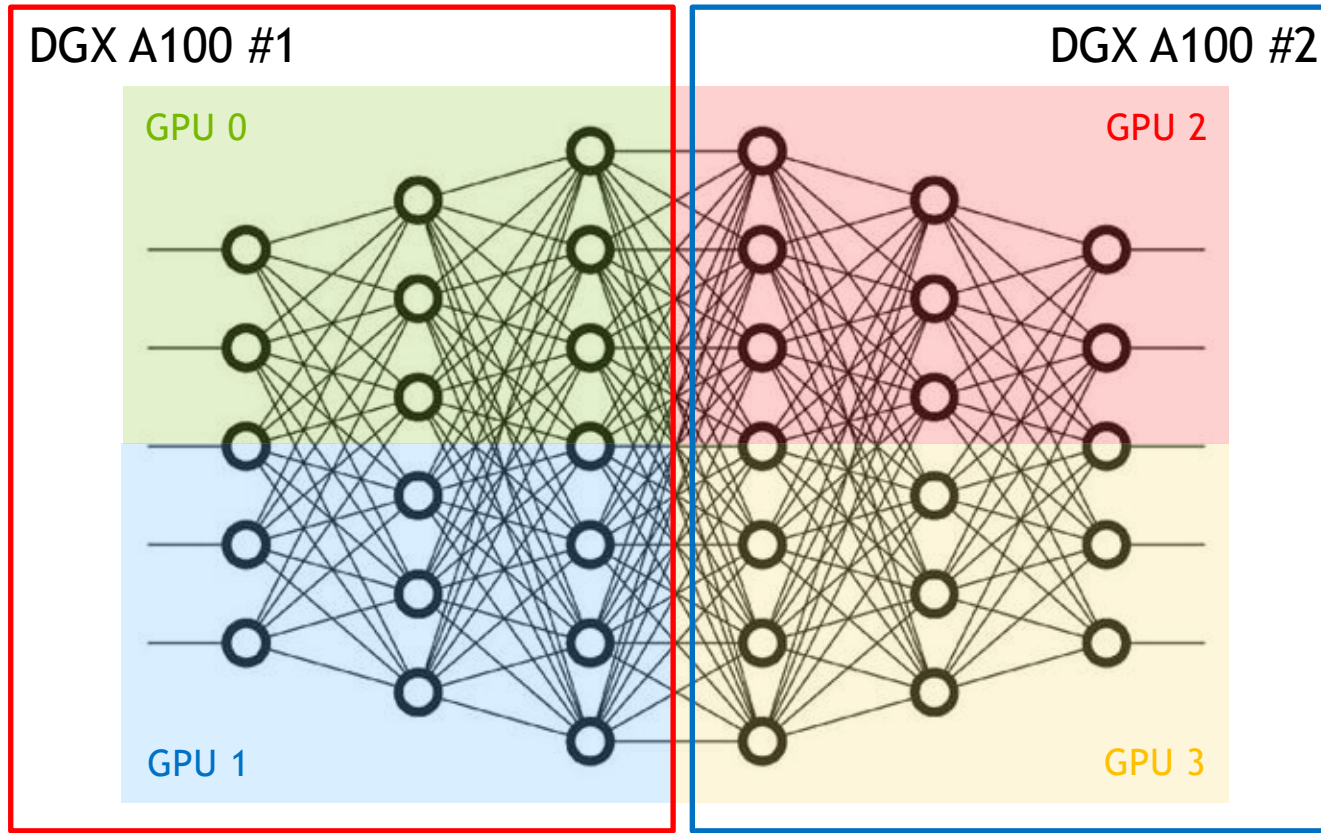


- Intra-Layer (Tensor) Parallelism
  - Split individual layers across multiple devices
  - **Inference:**
    - *Minimizes latency*



# MODEL PARALLELISM

Combined Model Parallelism. Multiple GPUs in Multiple DGXs.



Inter + Intra Parallelism



# TENSORRT TENSORRT-LLM

# NVIDIA TensorRT

## SDK for High-Performance Deep Learning Inference

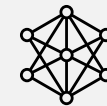
Optimize and deploy neural networks in production.

Maximize throughput for latency-critical apps with compiler and runtime.

Optimize every network, including CNNs, RNNs, and Transformers.

1. Reduced mixed precision: FP32, TF32, FP16, and INT8.
2. Layer and tensor fusion: Optimizes use of GPU memory bandwidth.
3. Kernel auto-tuning: Select best algorithm on target GPU.
4. Dynamic tensor memory: Deploy memory-efficient apps.
5. Multi-stream execution: Scalable design to process multiple streams.
6. Time fusion: Optimizes RNN over time steps.

<https://developer.nvidia.com/tensorrt>



Trained  
DNN



TensorRT  
Optimizer



TensorRT  
Runtime



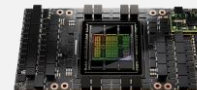
Data Center



Embedded



Application



Data Center  
GPUs



Jetson

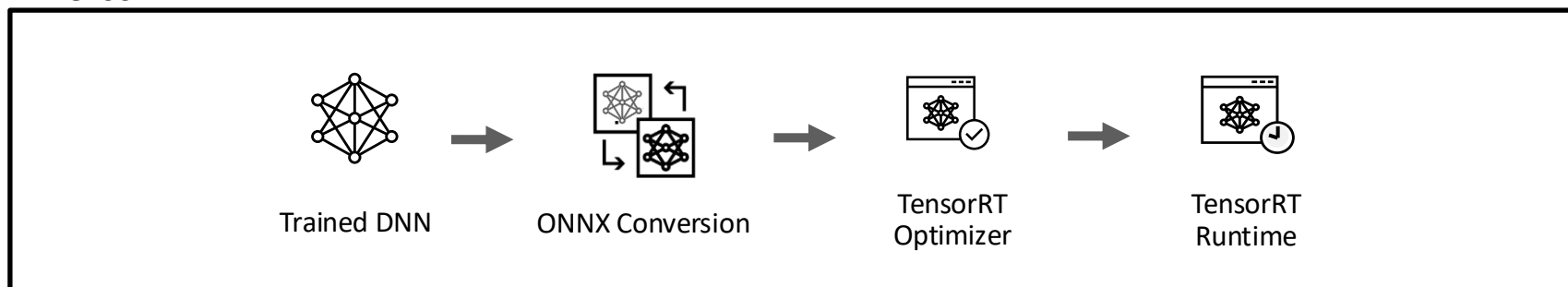


RTX GPUs

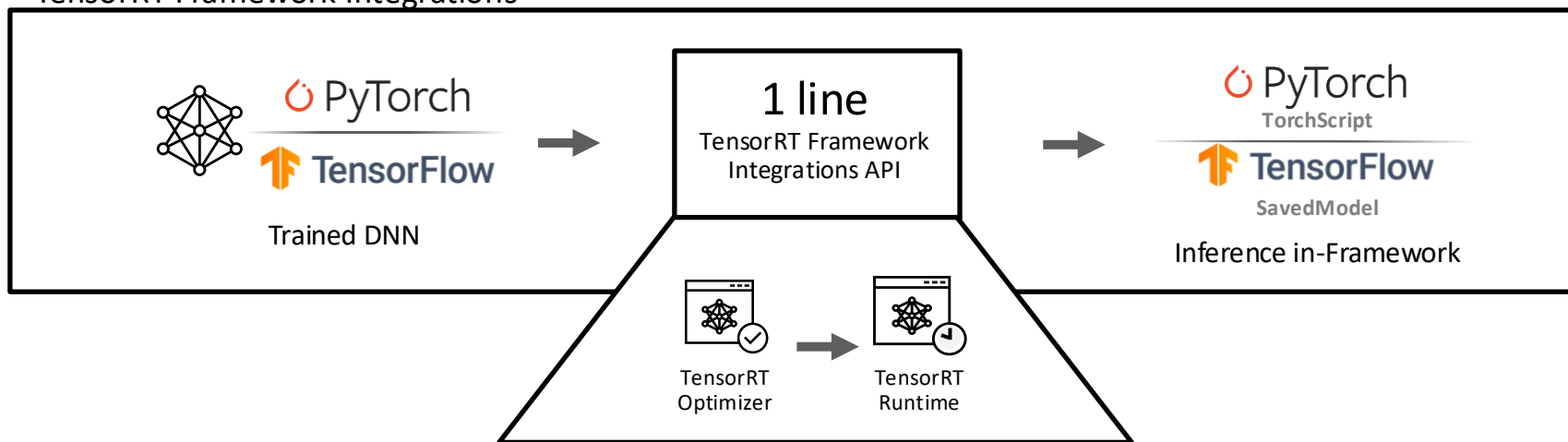


# Inference Optimization Workflow for TensorRT

## TensorRT



## TensorRT Framework Integrations



# TensorRT-LLM in the *DL Compiler* Ecosystem

TensorRT-LLM builds on TensorRT Compilation

- **TensorRT-LLM**

- Built *on-top of* TensorRT
  - Leverages TensorRT for general graph optimizations & fast kernels
- Adds LLM specific optimizations:
  - KV Caching & Custom MHA Kernels
  - Inflight batching, Paged KV Cache (Attention)
  - Multi-GPU, Multi-Node
  - & more
- *ONLY for LLMs*

- **TensorRT**

- General purpose Deep Learning Inference Compiler
  - Graph rewriting, constant folding, kernel fusion
  - Optimized GEMMs & pointwise kernels
  - Kernel Auto-Tuning
  - Memory Optimizations
  - & more
- *All AI Workloads*

## TensorRT-LLM

LLM specific optimizations:

- KV Caching
- Multi-GPU, Multi-Node
- Custom MHA optimizations
- Paged KV Cache (Attention)
- etc...

## TensorRT

General Purpose Compiler

- Optimized GEMMs & general kernels
- Kernel Fusion
- Auto Tuning
- Memory Optimizations
- Multi-stream execution

# TensorRT-LLM Execution

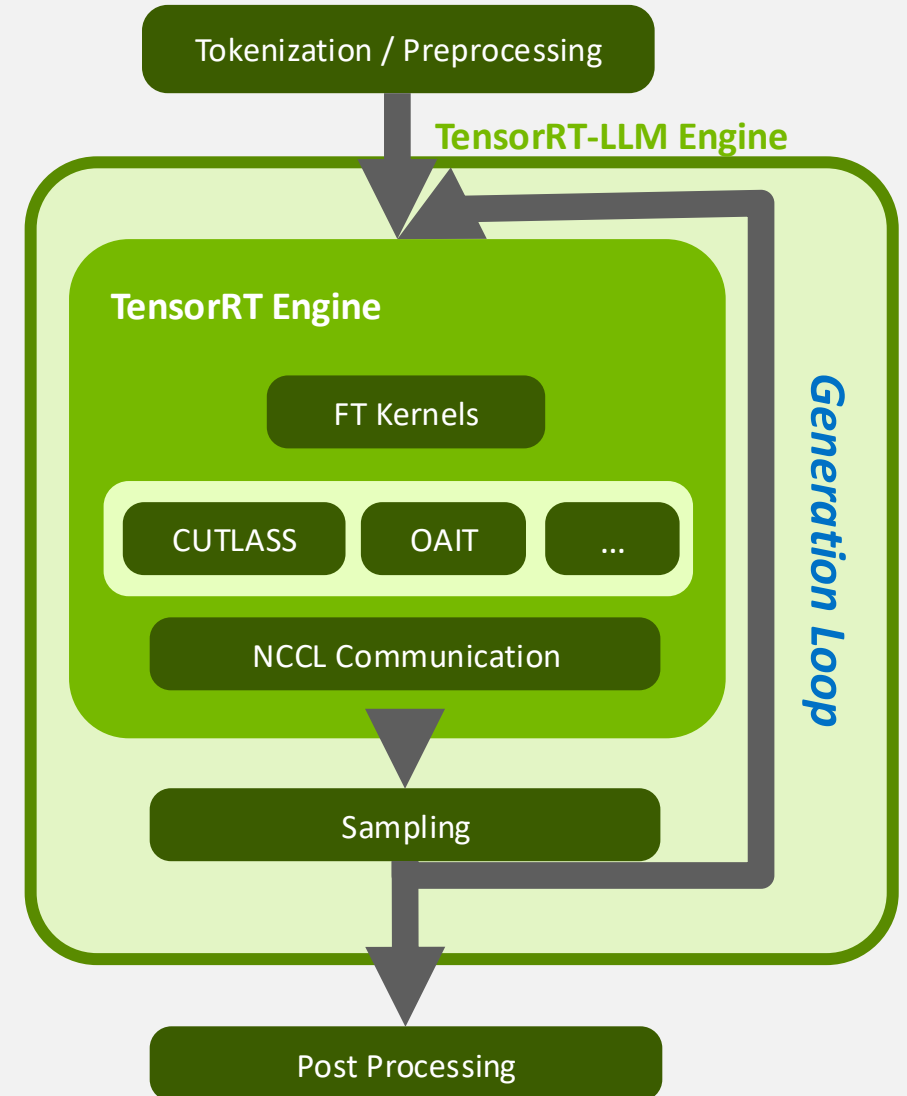
Built on top of FasterTransformer & TensorRT

TensorRT Engine accelerates the Transformer Architecture

- Executes both TensorRT compiled graph & Plugins
- Handles all plugin management & GPU communication

TensorRT-LLM handles the entire LLM

- Sampling runs on GPU for best performance loop
- Handles auto-regressive loop & batching
- Tokenization / detokenization can all be used directly

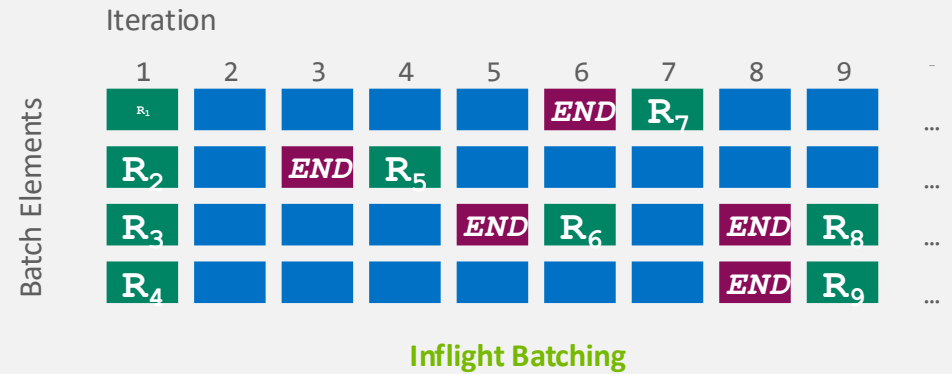
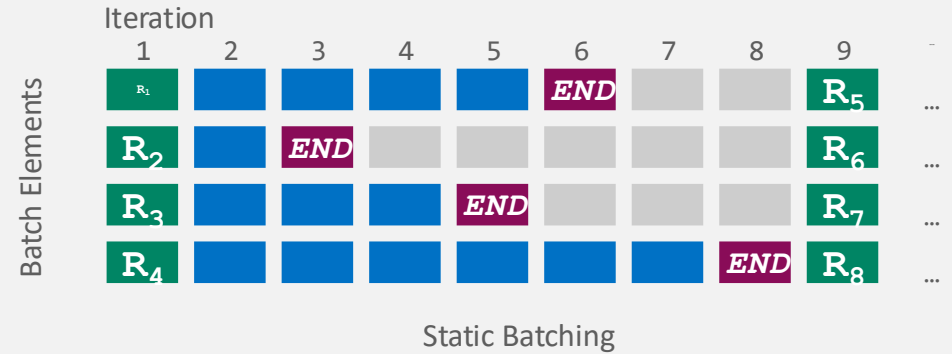


# Inflight Batching

## Maximizing GPU Utilization during LLM Serving

TensorRT-LLM provides custom Inflight Batching to optimize GPU utilization during LLM Serving

- Replaces completed requests in the batch
  - Evicts requests after EoS & inserts a new request
- Improves throughput, time to first token, & GPU utilization
- Integrated directly into the TensorRT-LLM Triton backend
- Accessible through the TensorRT-LLM Batch Manager



Context Gen EoS NoOp

# KV Cache Optimizations

## Paged & Quantized KV Cache

Paged KV Cache improves memory consumption & utilization

- Stores keys & values in non-contiguous memory space
- Allows for reduced memory consumption of KV cache
- Allocates memory on demand

Quantized KV Cache improves memory consumption & perf

- Reduces KV Cache elements from 16b to 8b (or less!)
- Reduces memory transfer improving performance
- Supports INT8 / FP8 KV Caches

Both allow for increased peak performance

KV Cache Contents:

TensorRT-LLM optimizes inference on NVIDIA GPUs ...

|         |          |        |           |           |
|---------|----------|--------|-----------|-----------|
| Block 0 | TensorRT | LLM    | optimizes | inference |
| Block 1 | on       | NVIDIA | GPUs      | ...       |
| Block 2 |          |        |           |           |
| Block 3 |          |        |           |           |

Traditional KV Caching

|                |          |        |           |           |
|----------------|----------|--------|-----------|-----------|
| B <sub>0</sub> | TensorRT | LLM    | optimizes | inference |
| B <sub>1</sub> |          |        |           |           |
| B <sub>2</sub> | on       | NVIDIA | GPUs      | ...       |
| B <sub>3</sub> |          |        |           |           |

Paged KV Cache

|                |        |     |        |        |    |        |      |     |
|----------------|--------|-----|--------|--------|----|--------|------|-----|
| B <sub>0</sub> | TRT... | LLM | opt... | inf... | on | NVIDIA | GPUs | ... |
| B <sub>1</sub> |        |     |        |        |    |        |      |     |
| B <sub>2</sub> |        |     |        |        |    |        |      |     |
| B <sub>3</sub> |        |     |        |        |    |        |      |     |

Quantized Paged KV Cache

Allocated Free

# Quantization

## Supported Precisions & Models

- Utilizes Hopper FP8 “Transformer Engine”
- Support many 8bit & 4bit methods
  - FP8, INT8/INT4 Weight only, INT8 Smooth Quant, AWQ, GPTQ
  - Support varies by model
- Reduced model size, memory bandwidth, & compute
  - Improves performance & allows for larger models per GPU
- Model optimization toolkit to quantize pre-trained models

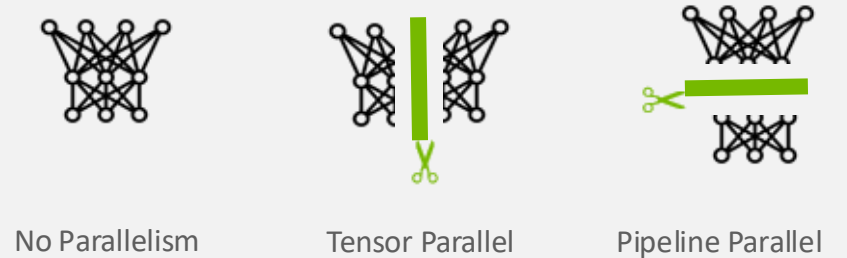
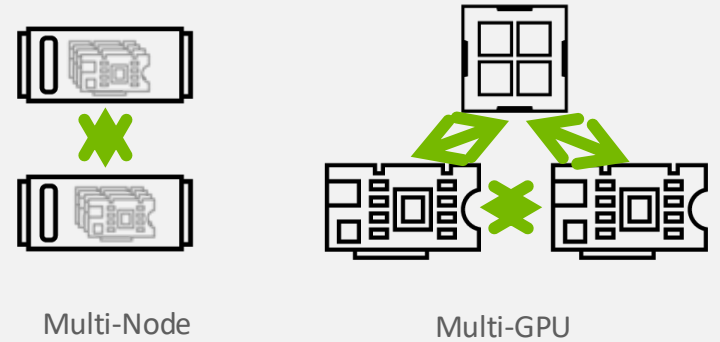
|                     | FP32 | FP16 | BF16 | FP8 | INT8 | INT4 |
|---------------------|------|------|------|-----|------|------|
| Volta (SM70)        | Y    | Y    | N    | N   | Y    | Y    |
| Turing (SM75)       | Y    | Y    | N    | N   | Y    | Y    |
| Ampere (SM80, SM86) | Y    | Y    | Y    | N   | Y    | Y    |
| Ada-Lovelace (SM89) | Y    | Y    | Y    | Y   | Y    | Y    |
| Hopper (SM90)       | Y    | Y    | Y    | Y   | Y    | Y    |

| Model      | FP32 | FP16 | BF16 | FP8 | W8A8 SQ | W8A16 | W4A16 | W4A16 AWQ | W4A16 GPTQ |
|------------|------|------|------|-----|---------|-------|-------|-----------|------------|
| Baichuan   | Y    | Y    | Y    | .   | .       | Y     | Y     | .         | .          |
| BERT       | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| BLOOM      | Y    | Y    | Y    | .   | Y       | Y     | Y     | .         | .          |
| ChatGLM    | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| ChatGLM-v2 | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| Falcon     | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| GPT        | Y    | Y    | Y    | Y   | Y       | Y     | Y     | .         | .          |
| GPT-J      | Y    | Y    | Y    | Y   | Y       | Y     | Y     | Y         | .          |
| GPT-NeMo   | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| GPT-NeoX   | Y    | Y    | Y    | .   | .       | .     | .     | .         | Y          |
| LLaMA      | Y    | Y    | Y    | .   | Y       | Y     | Y     | Y         | Y          |
| LLaMA-v2   | Y    | Y    | Y    | Y   | Y       | Y     | Y     | Y         | Y          |
| OPT        | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| SantaCoder | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |
| StarCoder  | Y    | Y    | Y    | .   | .       | .     | .     | .         | .          |

# Multi-GPU Multi-Node

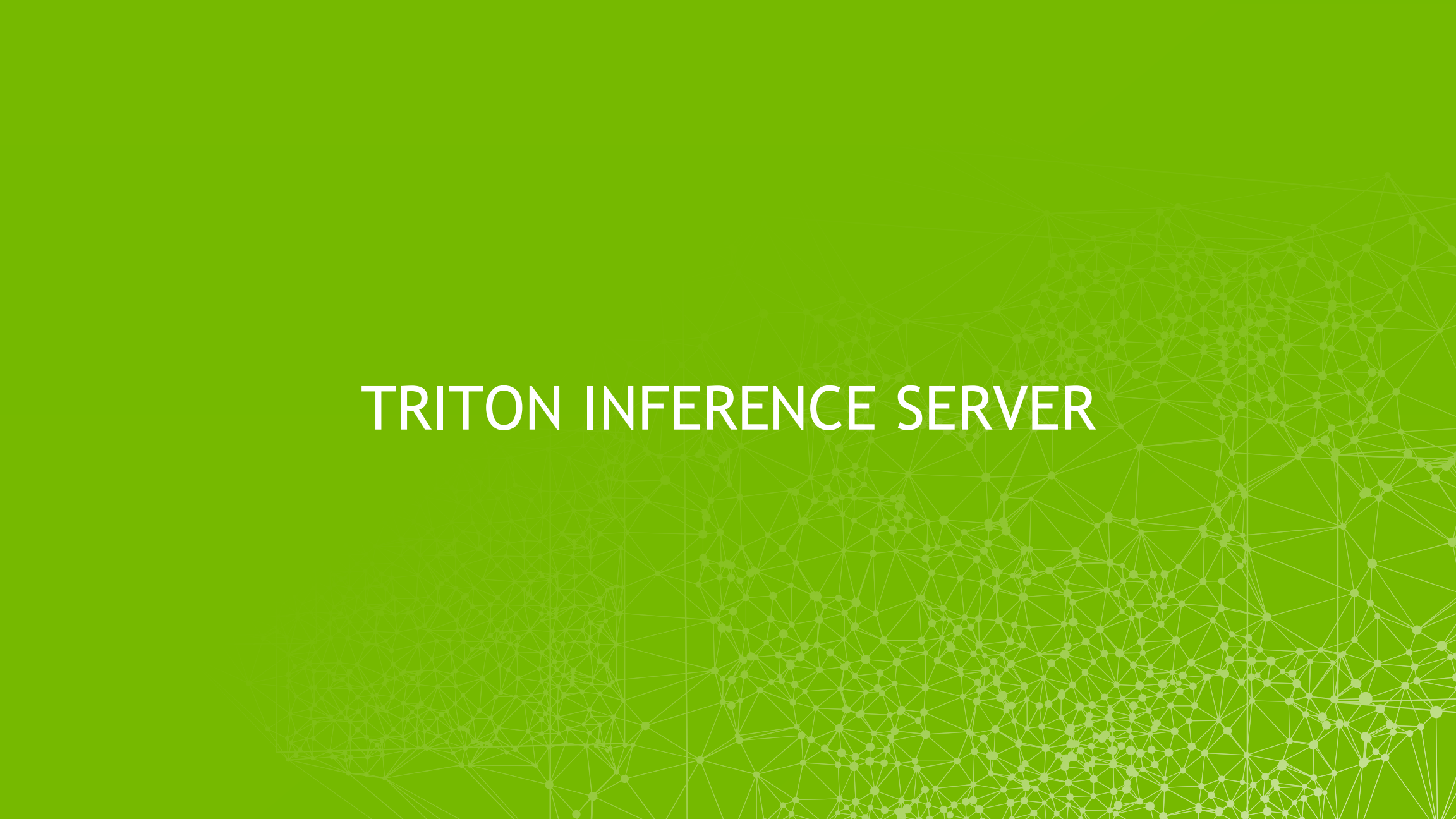
## Sharding Models across GPUs

- Supports Tensor & Pipeline parallelism
- Allows for running very large models (tested up to 530B)
- Supports multi-GPU (single node) & multi-node
- TensorRT-LLM handles communication between GPUs
- Examples are parametrized for sharding across GPUs





# TRITON INFERENCE SERVER



# TRITON INFERENCE SERVER

Triton Takes Care of Plumbing To Deploy Models for Inference

## Multiple Frameworks

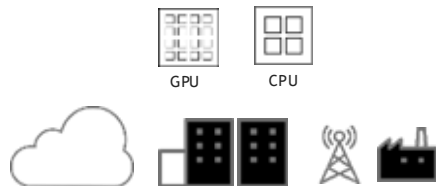


All Major Framework  
Backends For Flexibility &  
Consistency

Concurrent Model Execution  
For High Throughput &  
Utilization, lower TCO

Standard HTTP/gRPC  
Communication

## Inferencing on GPU and CPU



Inference Serving on GPU &  
CPU Across

Cloud | Data Center | Edge

Bare metal | Virtualization

## Different Types of Queries



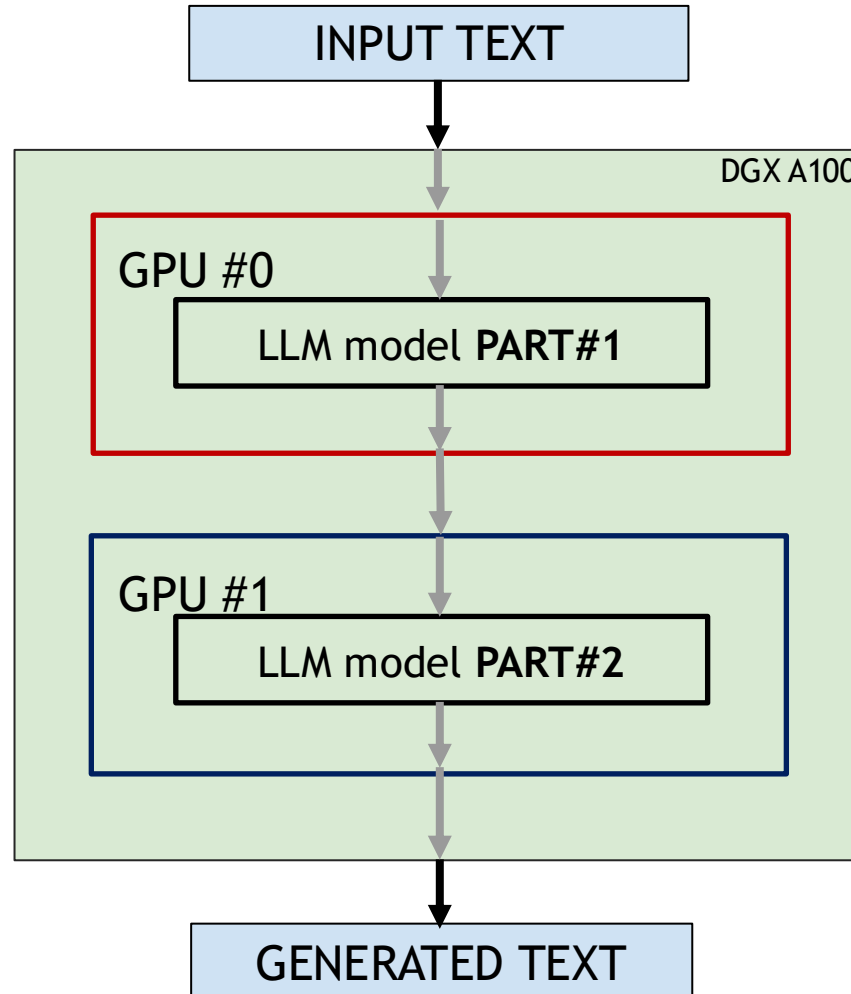
Support for Different Types  
of Inference Queries Used in  
Different Use Cases

## Dynamic Batching



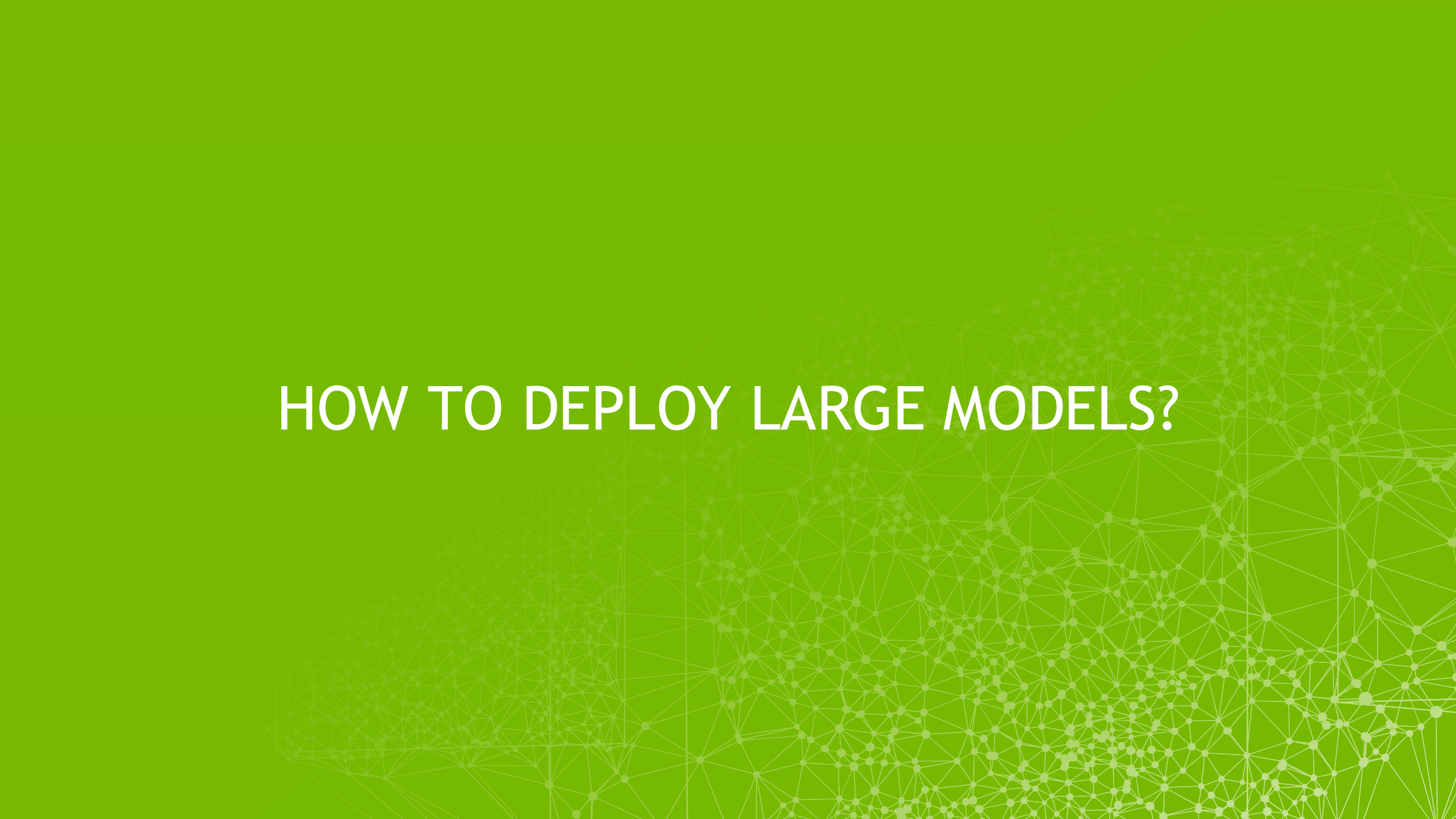
Dynamic Batching Maximizes  
Throughput Under Latency  
Constraint

# MODEL ENSEMBLING

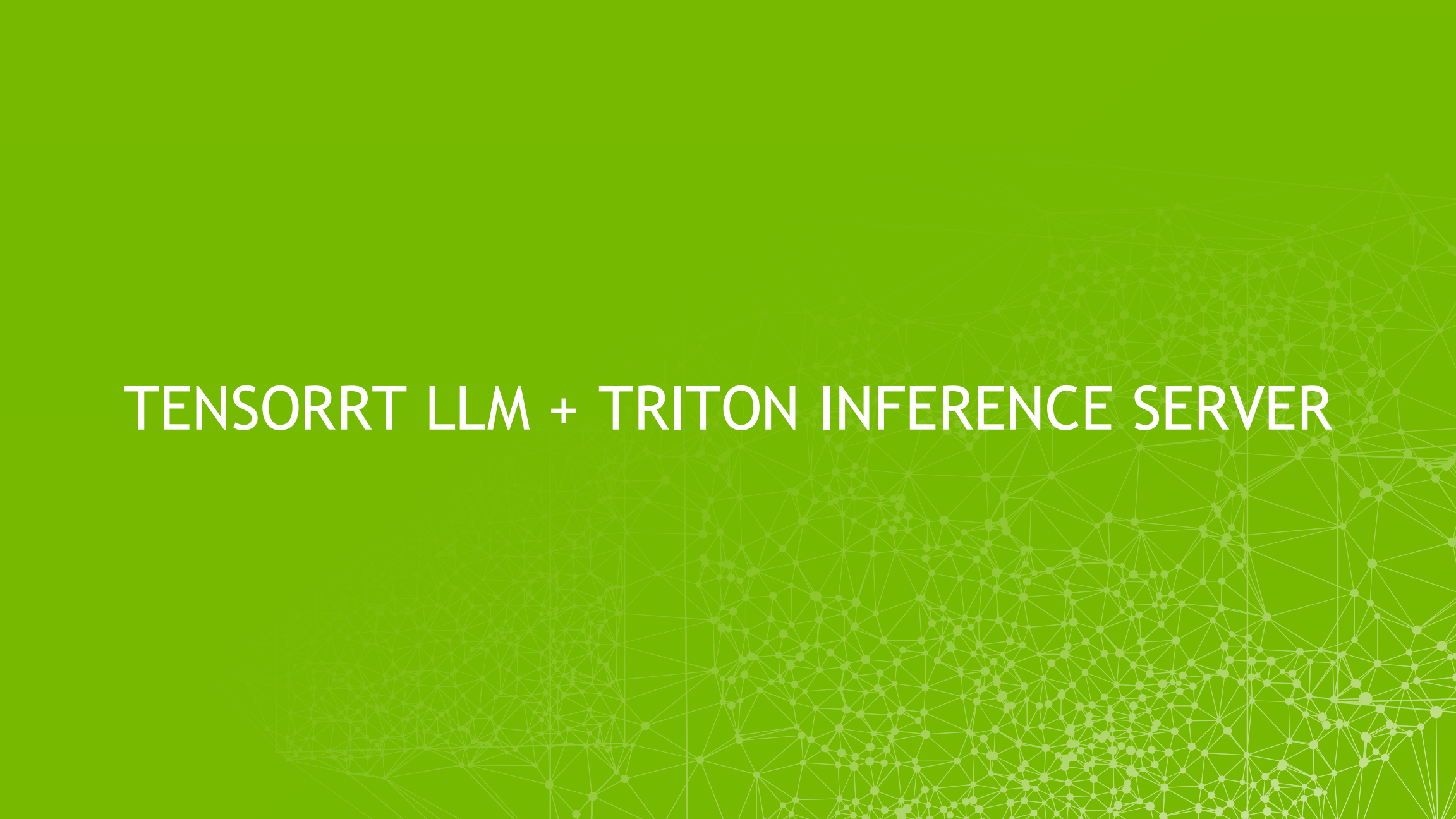


It's easy to create pipeline  
for parts of **one huge**  
model in TRITON

# HOW TO DEPLOY LARGE MODELS?

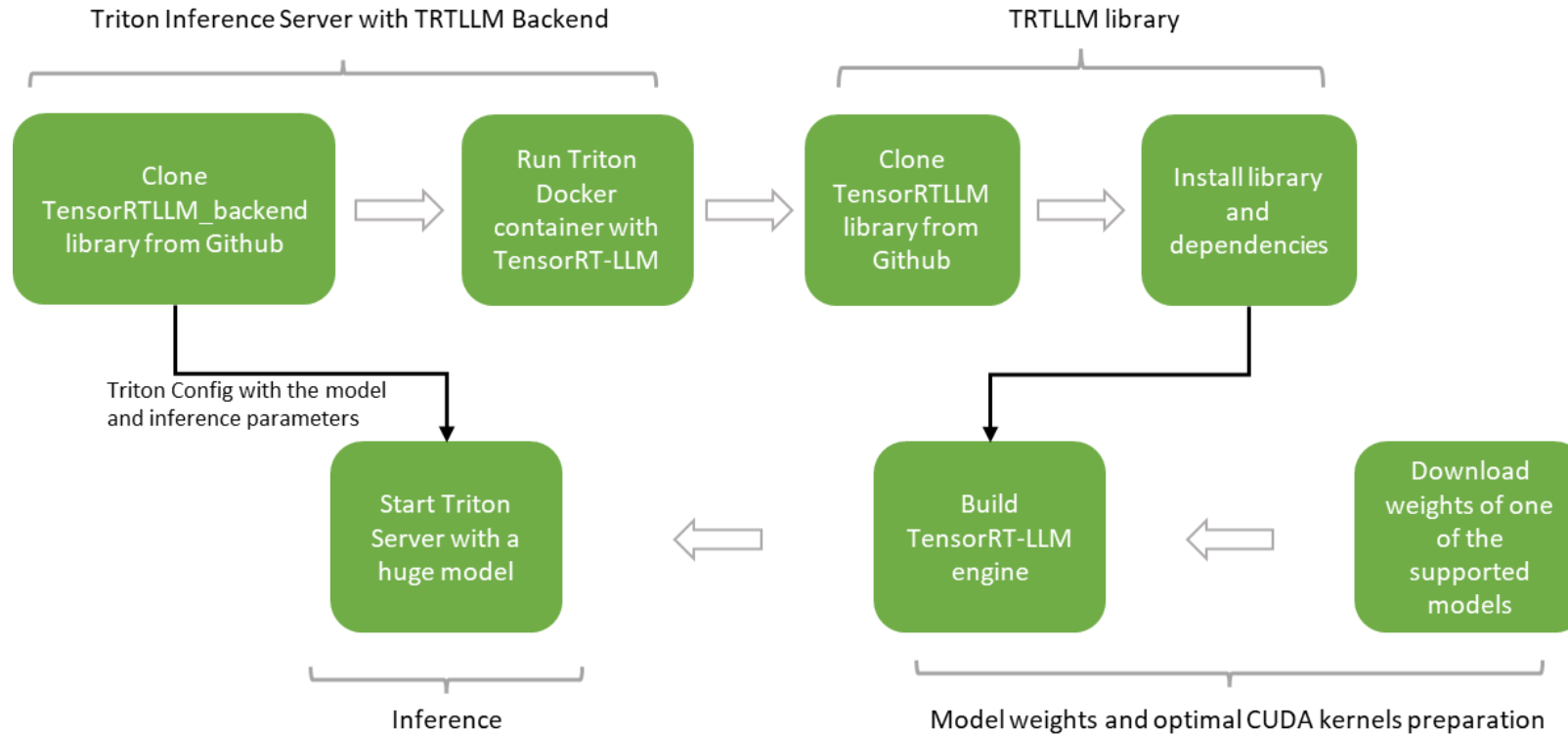


# TENSORRT LLM + TRITON INFERENCE SERVER

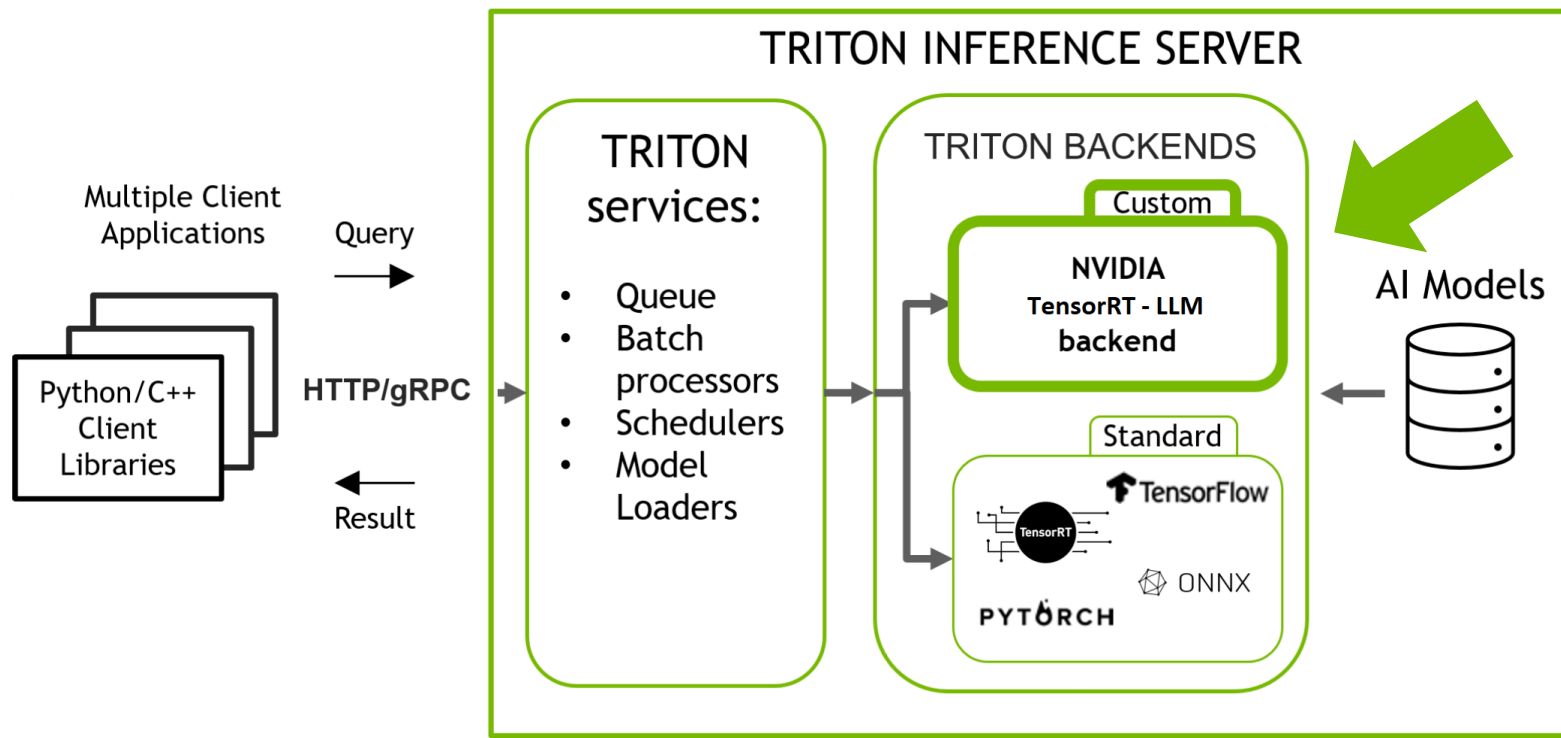
The background of the slide features a smooth gradient transitioning from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of white dots connected by thin white lines, resembling a neural network or a data mesh. The density of these connections increases towards the right side of the image.

# LARGE SCALE LLM DEPLOYMENT

## TensorRT LLM and Triton Inference Server

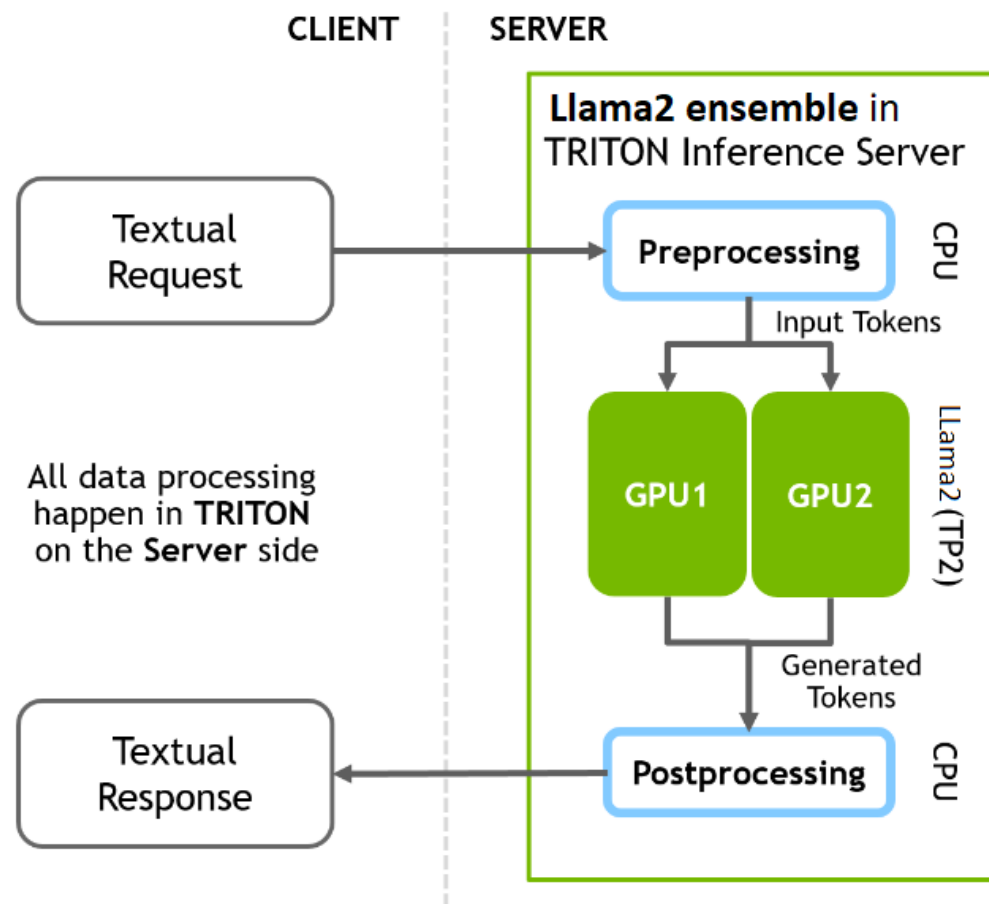


# OVERALL ARCHITECTURE





# LLAMA-2 TRITON ENSEMBLE



# TensorRT-LLM Performance Improvement

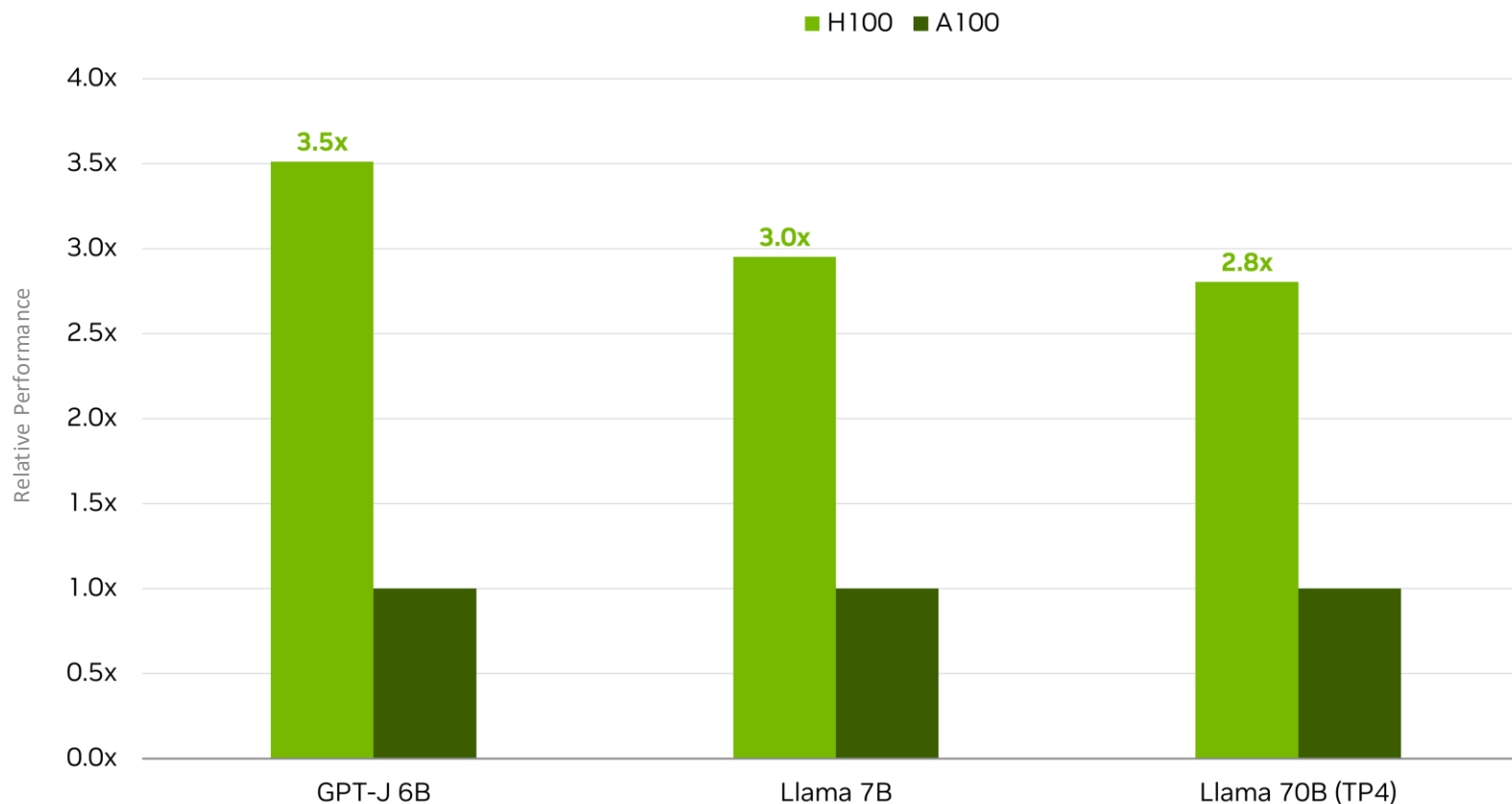
Up to 9x faster than baseline LLM implementations in DL frameworks



*TensorRT-LLM v0.5.0 internal build. HF Accelerate. Tokens/s/GPU relative improvement  
DGX H100. TensorRT-LLM FP8, HF Accelerate FP16.  
Max batchsize up to 64. Input & output sequence length 128:128  
TensorRT-LLM Llama 70B TP2, HF Accelerate PP4*

# TensorRT-LLM Performance Across Architectures

End-to-End Performance Using Inflight Batching & Triton



*TensorRT-LLM v0.5.0 internal build. Triton with TensorRT-LLM inflight batching backend  
DGX H100 FP8 & DGX A100 FP16  
CNN Daily Mail dataset. Varying max concurrency. SQL serving scenario  
TP4 = Tensor Parallel across 4 devices.*

# LAB OVERVIEW



# LAB

## Overview

- Baseline inference Of LLama2 with 13B parameters using the Hugging Face library and PyTorch
- Inference of LLama2 with TensorRT-LLM
- Distributed inference: Tensor Parallel (TP) and/or Pipeline Parallel (PP)
- Deployment of Llama2 with Triton Inference Server

